

EE351期中实验报告

12313505 胡伟毅

基本要求：

- 1.系统梳理并总结实验1至实验10的内容，形成完整的实验报告。
- 2.报告需图文并茂，结构清晰，条理分明。
- 3.清晰阐述每个实验的基本原理与核心功能，并详细说明各项基本功能的实现方法。
- 4.解释每个实验（实验1除外）的编程思路。附上所有自编程序的完整源代码（实验1无需提供源码）。
- 5.每个实验均需现场完成验收或在课程BB（Blackboard）系统中按时完成并成功提交对应的实验作业。

加分项：

- 1.为每个实验（实验1除外）编写的程序提供清晰的程序流程图或算法逻辑框图。
- 2.对于包含拓展任务的实验，在报告中详细说明其实现方法，可获得bonus加分。
- 3.其他在实验过程中展现的创新性、深入理解等老师认可的突出表现，也可获得bonus加分。

注：本报告中所有接线中的“树莓派GPIOxx引脚”指的均是T型转接板上的引脚编号（即BCM编码方式）

实验1：树莓派的系统安装、远程桌面登录

- 1.使用官方的Raspberry Pi Imager将系统烧录进SD卡
- 2.进行OS customization
- 3.用笔记本的热点将树莓派联网，便于查看ip地址，后续用SSH、VNC连接
- 4.用VNC和Vscode的SSH（后续编程调试更方便）分别进行连接，验证没问题，确定树莓派系统成功安装，且可以实现远程桌面登录。

实验2：编程基础及双色灯实验

- 1.首先学习了如何备份树莓派系统
- 2.学习树莓派有关的基础知识
 - (1) Linux常用命令

分类	命令	含义
目录切换	pwd	# 显示当前所在目录
	cd ~	#切换到主目录 (/home/pi) .~可以不写
	cd dir	#切换到指定目录，dir表示文件路径
	cd ..	#切换到上一级目录
文件展示	ls	#展示当前目录下所有的文件和文件夹（不包含隐藏文件）
	ls -a	#展示当前目录下所有的文件和文件夹（包含隐藏文件）
创建	touch file	#创建文件file
	mkdir dir	#创建目录dir
查看	cat file	#查看文件file内容
	more file	#查看文件file内容
	head file	#查看文件file前10行
	tail file	#查看文件file后10行
删除	rm file	#删除文件file
	rm -r dir	#删除目录dir
重命名或移动	mv file 1 file2	#若file2不存在，则将file1改名为file2；若file2存在，则file1覆盖掉原file2，并更名为file2
	mv file dir	#将文件file移动到dir目录中

(2) 编程基础

C/C++编程需要安装 Wiring Pi 库，Python编程使用 mu code （一种代码编辑器）。

由于我后期选择用SSH连接到树莓派进行编程，所以我直接在笔记本上下载了vscode，在vscode里直接用python进行编程。

(3) 文本编辑器

文本编辑器	nano	vi	
基本用法	1. 打开 / 创建文件：nano 文件名（如 nano test.txt） 2. 编辑：直接输入文字（默认进入编辑模式） 3. 保存：Ctrl + O（回车确认文件名） 4. 退出：Ctrl + X	1. 打开 / 创建文件：vi 文件名（如 vi test.txt） 2. 进入编辑：命令模式下按 i（插入模式） 3. 保存退出：按 Esc 回命令模式，输入 :wq 并回车 4. 放弃退出：命令模式下输入 :q! 并回车	
常用快捷键	- Ctrl + O：保存文件 - Ctrl + X：退出编辑器 - Ctrl + K：剪切当前行 - Ctrl + U：粘贴剪切内容 - Ctrl + W：搜索文本 - Ctrl + \：替换文本 - Ctrl + C：显示光标位置	命令模式	末行模式
		- i：进入插入模式（在光标前编辑） - dd：删除当前行 - yy：复制当前行 - p：粘贴内容 - u：撤销操作 - Shift + zz：快速保存并退出 - h/j/k/l：左 / 下 / 上 / 右移动光标	- :w：保存文件 - :q：退出（未保存时提示） - :q!：强制退出（不保存） - :wq：保存并退出 - :set nu：显示行号 - :set nonu：隐藏行号

在vscode内进行编程，就直接使用软件自带的操作了，不使用文本编辑器。

(4) 硬件基础（三种编码方式）

- BOARD板上引脚号
- BCM（python编程使用）
- wiringPi（C编程使用）

查看引脚编码的指令（`gpio pinout`/`gpio readall`），后者必须要安装WiringPi库

`pinout` 指令的作用

当你在树莓派的终端中输入 `pinout` 并回车后，它会输出以下关键信息：

- **ASCII图示**：一个彩色的ASCII艺术图，直观地展示了你的树莓派型号、主要芯片的位置（如SoC、RAM）以及40个GPIO引脚的物理排列。^{[1][2]}
- **引脚编号**：它会同时列出每个引脚的两种主要编号方式：
 - **物理编号（Board）**：按照引脚在电路板上的物理位置从1到40进行编号。
 - **BCM编号（GPIO）**：这是Broadcom芯片的GPIO编号，也是Python编程中（如使用Rpi.GPIO或GPIO Zero库）最常使用的编号方式。
- **引脚功能**：清晰地标明了每个引脚的默认或特殊功能，例如：
 - **电源引脚**：3.3V、5V。
 - **地线引脚**：GND。
 - **通用输入/输出（GPIO）**：可以由用户编程控制的引脚。
 - **特殊功能引脚**：用于I2C、SPI、UART等通信协议的引脚。^{[3][4]}
- **硬件信息**：除了引脚信息，`pinout` 指令还会显示当前树莓派的硬件详情，包括型号、版本、SoC（片上系统）类型、RAM大小等。^[2]
- **参考网站**：输出信息的末尾通常会附上一个网址 <https://pinout.xyz/>，这是一个非常详尽的在线引脚图参考网站。^[2]

安装完成后，你就可以使用 `gpio readall` 指令了。

总结对比

特性	<code>pinout</code>	<code>readall</code> (旧)	<code>gpio readall</code> (新)
主要功能	显示引脚布局和功能定义	显示引脚实时状态（模式、电平）	显示引脚实时状态（模式、电平）
所属库	GPIO Zero	WiringPi	rpi-gpio
当前状态	预装在桌面版系统中，官方推荐	已废弃，在新系统中不再可用	官方推荐的替代方案，可能需手动安装
输出形式	彩色ASCII图，静态信息	表格，动态实时信息	表格，动态实时信息
使用场景	查看引脚定义和编号，用于编程参考	调试电路，检查引脚的输入输出电平	调试电路，检查引脚的输入输出电平

3.LED双色灯实验

实验目的：

实验意图就是一个简单的点灯程序，通过使用双色LED模块，完成红绿灯交替闪烁。

实验原理：

- 1.双色LED模块内置两个颜色的灯管，本实验是红色和绿色。
- 2.控制与灯管相连的GPIO引脚，可以选择点亮哪种颜色的灯管。
- 3.交替控制就可以实现红绿双色灯的闪烁

实验过程：

- (1) 将LED模块的GND引脚连接到树莓派（BCM编码方式）的GND引脚，两个颜色的灯管分别连接到GPIO19和GPIO26引脚。
- (2) 编写程序代码：将两个连接灯管的GPIO口配置为输出模式，并循环控制两个引脚状态，实现红绿双色的交替闪烁效果。

程序框图：



程序代码：

```
import RPi.GPIO as GPIO
import time

# 两个LED灯的GPIO口
led1 = 19 # 第一个颜色
led2 = 26 # 第二个颜色

GPIO.setmode(GPIO.BCM)
GPIO.setup(led1, GPIO.OUT)
GPIO.setup(led2, GPIO.OUT)

try:
    while True:
        # led1亮, led2灭
        GPIO.output(led1, GPIO.HIGH)
        GPIO.output(led2, GPIO.LOW)
        print("LED1 ON, LED2 OFF")
        time.sleep(1)
        # led1灭, led2亮
        GPIO.output(led1, GPIO.LOW)
        GPIO.output(led2, GPIO.HIGH)
        print("LED1 OFF, LED2 ON")
        time.sleep(1)
except KeyboardInterrupt:
    print("End")
finally:
    try:
        GPIO.cleanup()
        print("The program is over.GPIO has been cleaned")
    except Exception as e:
        print(f"GPIO cleanup failed: {e}")
```

实验3:开关控制LED灯

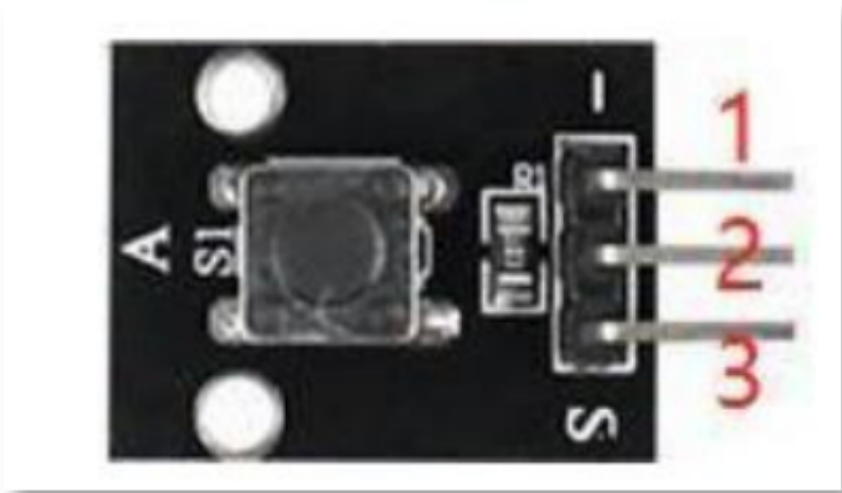
实验目的：

将轻触开关作为输入设备，实现通过树莓派检测按键状态，并通过按下按键完成LED灯模式的切换。

实验原理：

- 1. 轻触开关模块内部有一个轻触开关（按键开关）。
- 2. 当按下按键时，S 引脚输出低电平。
- 3. 当松开按键时，S 引脚输出高电平。

在本实验中，我们将使用树莓派的 GPIO 口检测轻触开关的 S 引脚状态，并使用 LED 灯指示按键的状态。



按键开关

实验过程：

- (1) 将红色灯管接到GPIO26引脚，绿色灯管接到GPIO19引脚；按键的S脚接到GPIO25引脚，VCC引脚接到树莓派的5V脚，GND引脚接到树莓派的GND引脚。
- (2) 配置GPIO引脚为输出模式，检测S引脚的状态，检测到低电平时，认定此时开关被按下。
- (3) 设置状态机，每次开关被按下就进入循环模式，分别是：红灯常亮，红灯闪烁，绿灯常亮，绿灯闪烁。

程序框图：



程序代码：

```
import RPi.GPIO as GPIO
import time
```

```

led_red = 26
led_green = 19
button = 25

GPIO.setmode(GPIO.BCM)
GPIO.setup(led_red, GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(led_green, GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(button, GPIO.IN, pull_up_down=GPIO.PUD_UP)

state = -1
state_start_time = time.monotonic()

stable_button_state = GPIO.input(button)
last_raw_state = stable_button_state
last_change_time = time.monotonic()
DEBOUNCE_SEC = 0.03

BLINK_INTERVAL = 0.5

try:
    while True:
        now = time.monotonic()
        raw = GPIO.input(button)
        if raw != last_raw_state:
            last_raw_state = raw
            last_change_time = now
        else:
            if raw != stable_button_state and (now - last_change_time) >= DEBOUNCE_SEC:
                stable_button_state = raw
                if stable_button_state == GPIO.LOW:
                    if state == -1:
                        state = 0
                    else:
                        state = (state + 1) % 4
                state_start_time = now
                print(f"Button press -> state {state}")
            if state == -1:
                GPIO.output(led_red, GPIO.LOW)
                GPIO.output(led_green, GPIO.LOW)
            elif state == 0:
                GPIO.output(led_red, GPIO.HIGH)
                GPIO.output(led_green, GPIO.LOW)
            elif state == 1:
                phase = int(((now - state_start_time) / BLINK_INTERVAL)) % 2
                GPIO.output(led_red, GPIO.HIGH if phase == 0 else GPIO.LOW)
                GPIO.output(led_green, GPIO.LOW)
            elif state == 2:
                GPIO.output(led_red, GPIO.LOW)
                GPIO.output(led_green, GPIO.HIGH)
            elif state == 3:
                phase = int(((now - state_start_time) / BLINK_INTERVAL)) % 2

```

```
GPIO.output(led_red, GPIO.LOW)
GPIO.output(led_green, GPIO.HIGH if phase == 0 else GPIO.LOW)
time.sleep(0.005)
except KeyboardInterrupt:
    print("End")
finally:
    GPIO.cleanup()
    print("The program is over. GPIO has been cleaned")
```

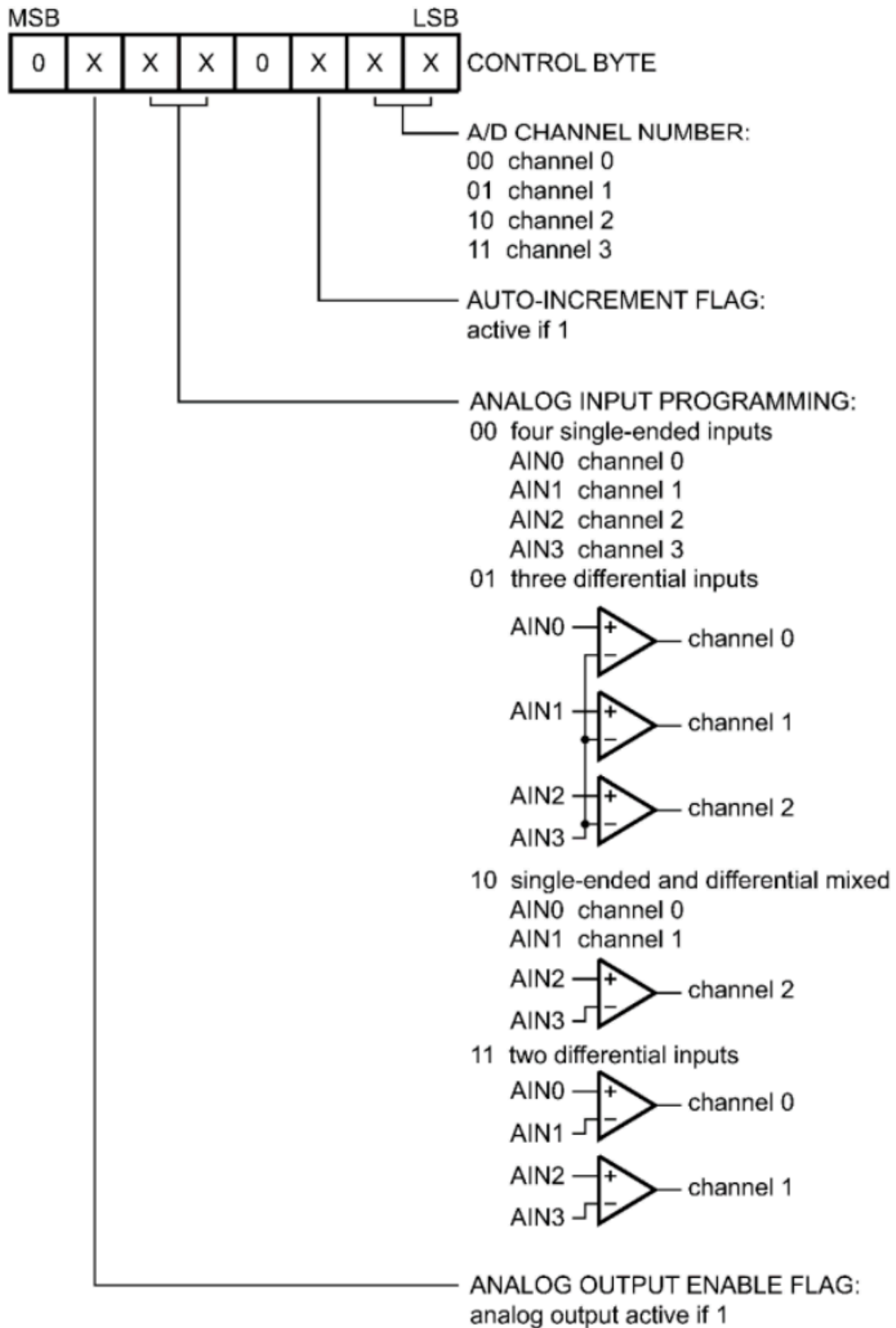
实验4:数模转换器

实验目的：

通过使用PCF8591模数转换器，控制灯的亮度，理解PCF8591的特性和工作原理。并掌握I2C通信协议，实现与PCF8591的通信。

实验原理：

- (1) PCF8591模数转换器的工作原理：I2C总线中的每个PCF8591设备，通过发送有效地址而被激活，这里的“地址”由固定部分和可编程部分组成。可编程部分由地址引脚A0、A1、A2。
- (2) 在I2C总线协议中，地址字节的最后一个位是读/写位，用于设置数据传输的方向。
- (3) 发送到PCF8591的第二个字节会被存储在控制寄存器中，寄存器中的高半字节用于使能模拟输出，并将模拟输入改为单端或者差分输入；寄存器中的低半字节则用于选择模拟输入通道。
- (4) 在本实验中，AIN0（模拟输入 0）端口用于接收电位器模块的模拟信号，AOUT）用于将模拟信号输出到LED 模块，以改变 LED 的亮度。
- (5) PCF8591模块自带光电二极管和NTC热敏电阻，用于光强感知和温度感知（在后续实验中有用到）。



实验过程：

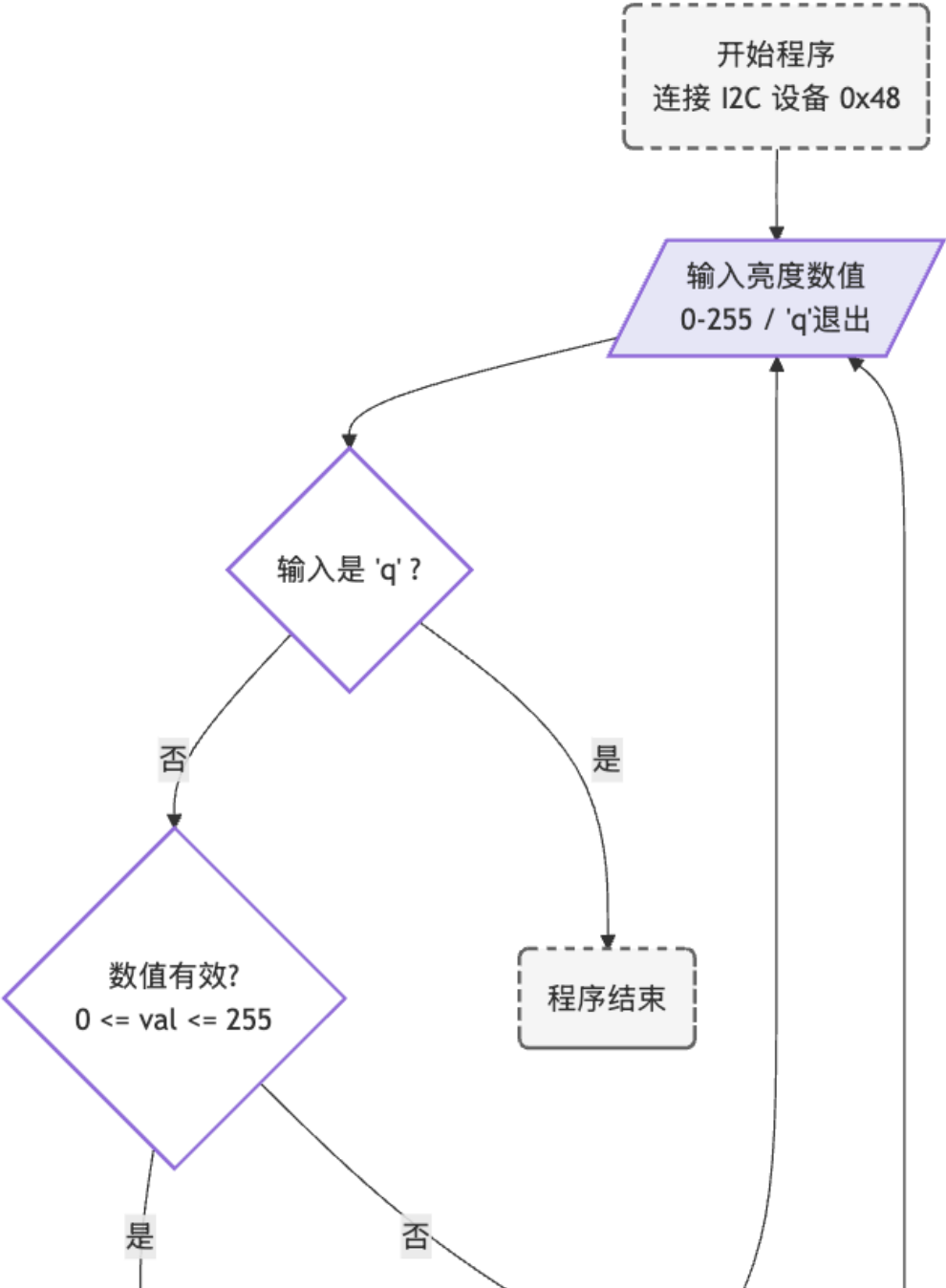
首先将PCF8591模块连接到I2C总线（PCF8591芯片的SDA引脚和SCL引脚与树莓派的对应引脚相连接），并设置好PCF8591的硬件地址为0x48，这个地址是树莓派通过总线寻找芯片的途径。

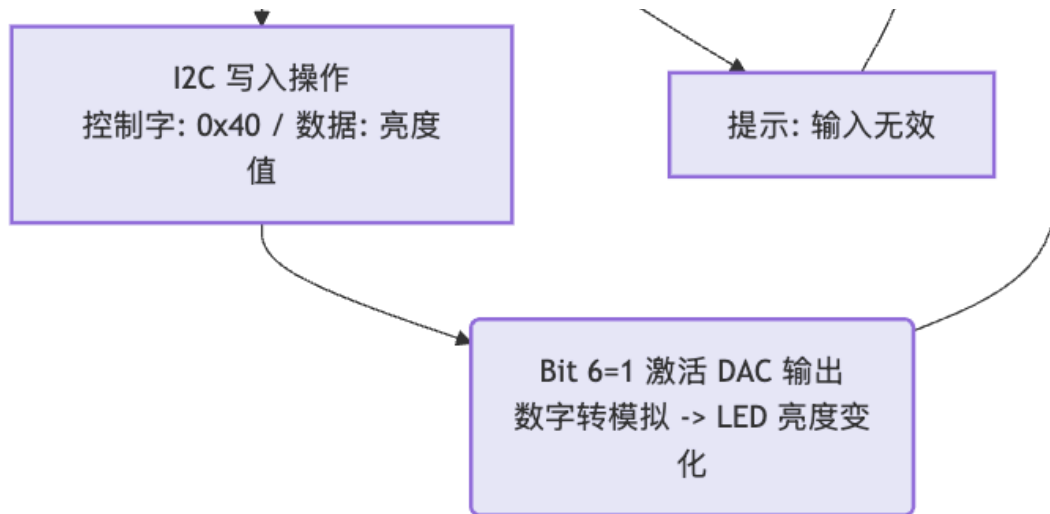
（1）任务：键盘输入LED亮度值，控制LED亮度（DA）

实验思路：

配置芯片为0x40（输出开关），0x40所对应的二进制代码中，Bit6所对应的是1，也就是使能输出，允许芯片向外输出电压。因此可以实现读取键盘输入的数字值，转为向外输出的模拟电压值。

程序框图：





程序代码：

```
import smbus

bus = smbus.SMBus(1)
DEVICE_ADDRESS = 0x48 # PCF8591 I2C address
CONTROL_BYTE = 0x40 # Enable DAC output

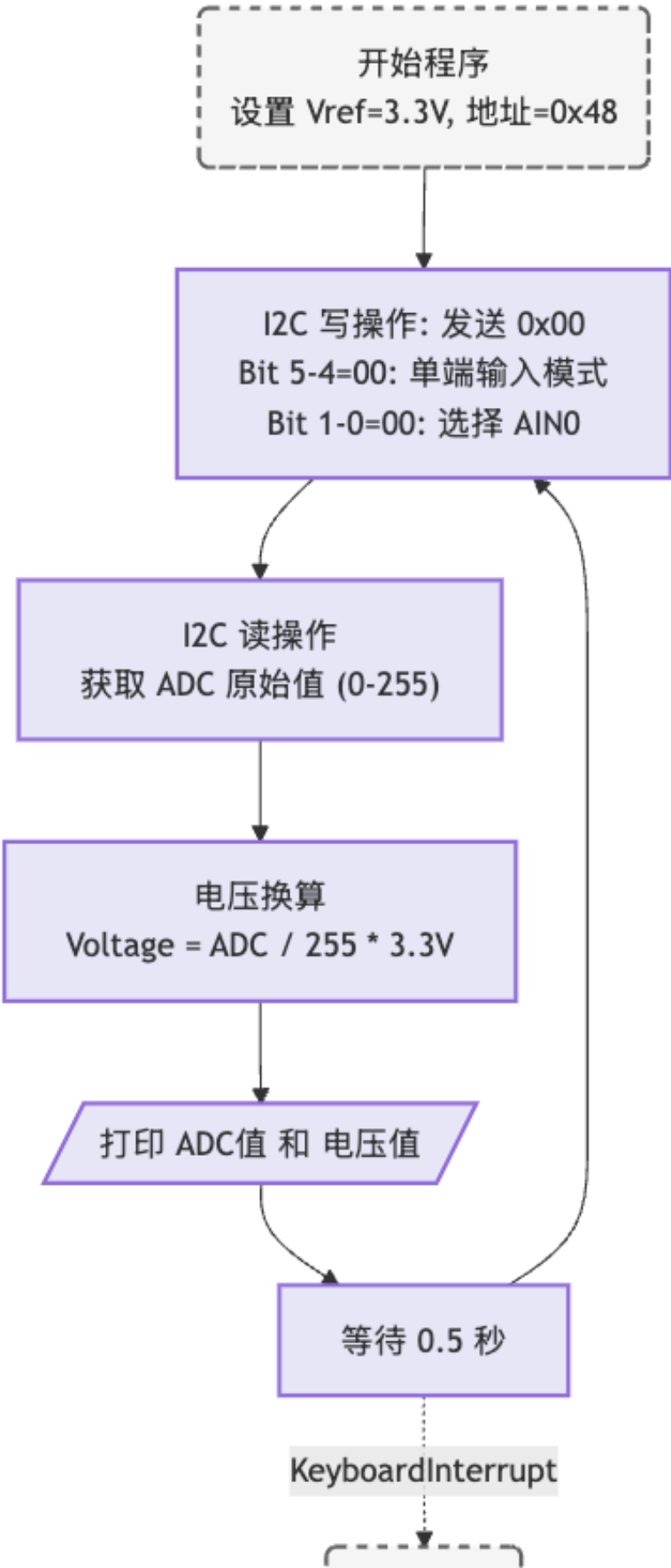
try:
    while True:
        value = input("Enter LED brightness (0-255, q to quit): ")
        if value.lower() == 'q':
            break
        try:
            brightness = int(value)
            if 0 <= brightness <= 255:
                bus.write_byte_data(DEVICE_ADDRESS, CONTROL_BYTE, brightness)
                print(f"Set brightness to {brightness}")
            else:
                print("Value must be between 0 and 255.")
        except ValueError:
            print("Invalid input. Please enter a number between 0 and 255 or 'q' to quit.")
    except KeyboardInterrupt:
        print("End")
```

(2) 任务：调节电位器，打印当前输入电压值（AD）

实验思路：

由于接线时将模数转换器的VCC接到了树莓派的3.3V引脚，所以这里将参考电压Vref设置为3.3V。将LED灯引脚接到PCF8591的AIN0引脚，根据控制字节，此时Bit5-4均需设置为0，才能实现芯片的“读取模式”，即将输入端设置成0x00，可以读取AIN0处进入的电压。

程序框图：



程序结束

程序代码：

```
import smbus
import time

bus = smbus.SMBus(1)
DEVICE_ADDRESS = 0x48
VREF = 3.3
control = 0x00

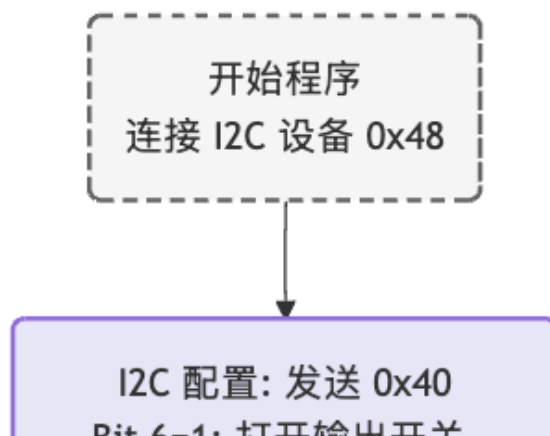
try:
    while True:
        bus.write_byte(DEVICE_ADDRESS, control)
        bus.read_byte(DEVICE_ADDRESS)
        adc = bus.read_byte(DEVICE_ADDRESS)
        voltage = (adc / 255.0) * VREF
        print(f"ADC: {adc:3d}  Voltage: {voltage:.2f}V")
        time.sleep(0.5)
except KeyboardInterrupt:
    print("stop")
```

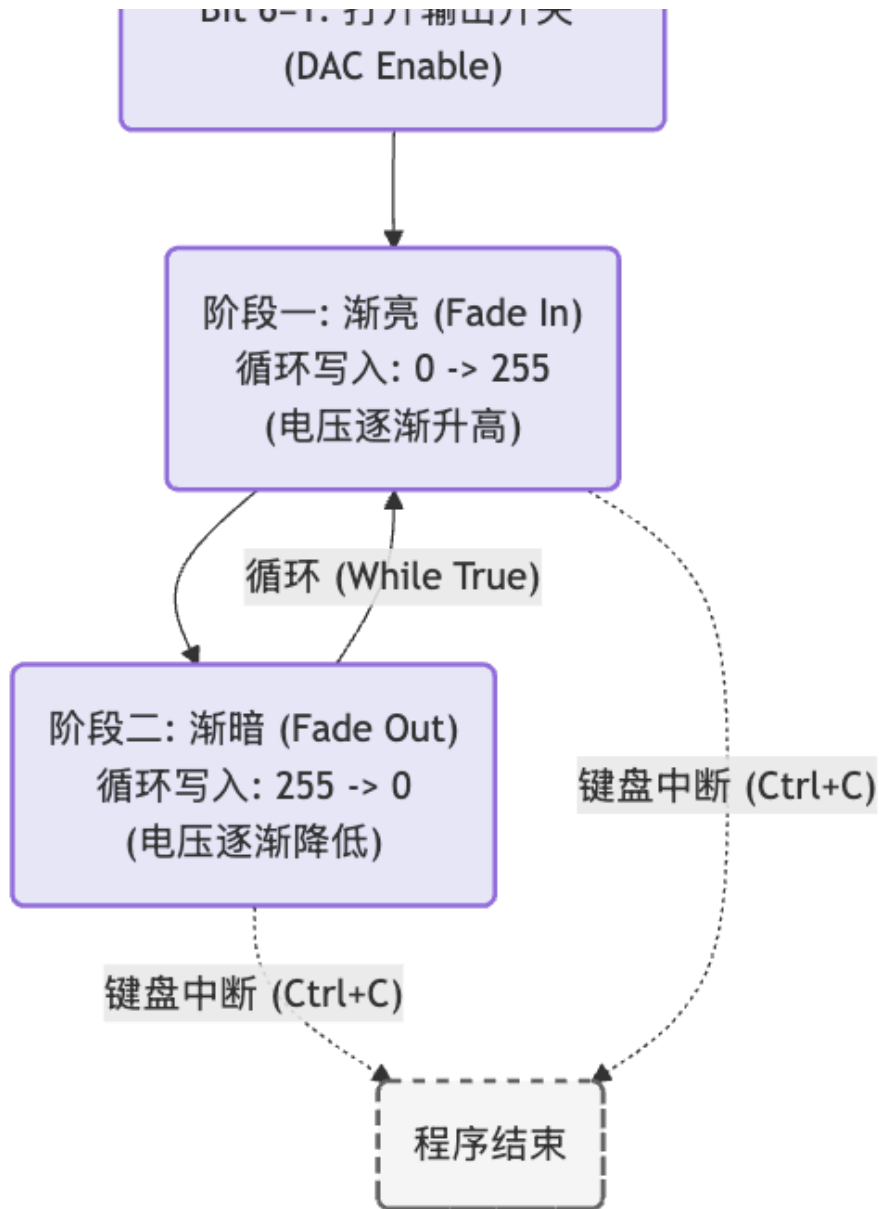
(3) 任务：呼吸灯（DA）

实验思路：

呼吸灯实际上就是不断调节输出的电压，所以要将输出开关打开，控制字节设置为0x40，芯片可以向LED灯不断发送不同的电压，实现呼吸灯效果。

程序框图：





程序代码:

```
import smbus
import time

bus = smbus.SMBus(1)
DEVICE_ADDRESS = 0x48 # PCF8591 I2C address
CONTROL_BYTE = 0x40 # Enable DAC output

try:
    while True:
        # Fade in
        for brightness in range(0, 256):
            bus.write_byte_data(DEVICE_ADDRESS, CONTROL_BYTE, brightness)
```

```

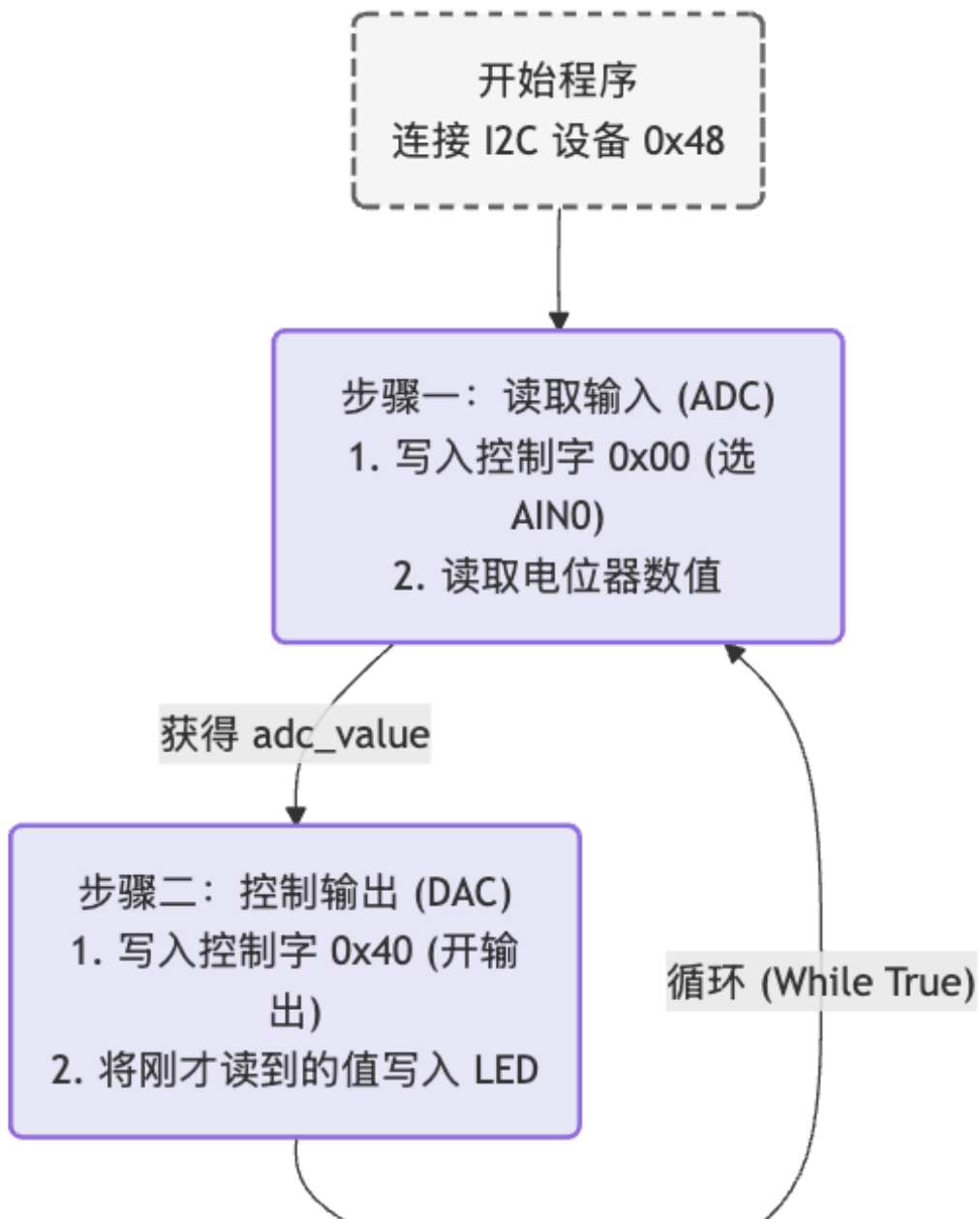
        time.sleep(0.01)
    # Fade out
    for brightness in range(255, -1, -1):
        bus.write_byte_data(DEVICE_ADDRESS, CONTROL_BYTE, brightness)
        time.sleep(0.01)
except KeyboardInterrupt:
    print("End")

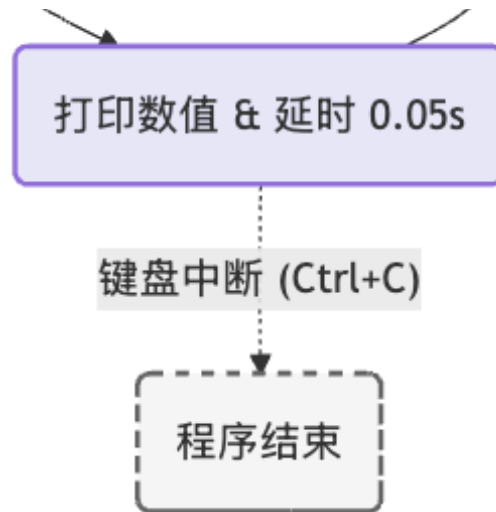
```

(4) 任务：调节电位器改变小灯泡亮度 (AD—DA)

实验思路：本题实际上可以分为两个操作：从电位器读取模拟电压；向LED灯输出电压。所以需要同时设置两个字节，分别为0x00和0x40，实现从AIN0读取电压，再输出给LED灯这一目标。

程序框图：





程序源码：

```
import smbus
import time

bus = smbus.SMBus(1)
DEVICE_ADDRESS = 0x48 # PCF8591 I2C address
CONTROL_BYTE_ADC = 0x00 # Select channel 0 (A0)
CONTROL_BYTE_DAC = 0x40 # Enable DAC output

try:
    while True:
        bus.write_byte(DEVICE_ADDRESS, CONTROL_BYTE_ADC)
        time.sleep(0.01)
        bus.read_byte(DEVICE_ADDRESS)
        adc_value = bus.read_byte(DEVICE_ADDRESS)
        bus.write_byte_data(DEVICE_ADDRESS, CONTROL_BYTE_DAC, adc_value)
        print(f"Potentiometer ADC: {adc_value}, LED brightness: {adc_value}")
        time.sleep(0.05)
except KeyboardInterrupt:
    print("End")
```

实验5:模拟温度传感器

实验目的：

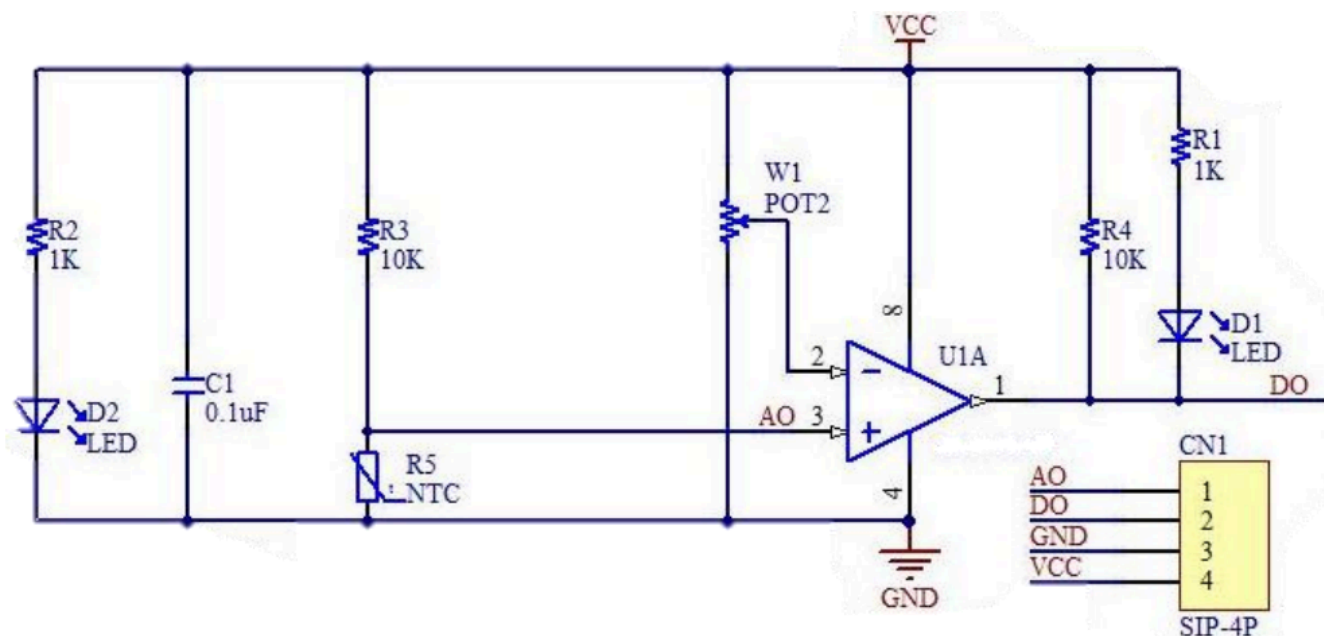
温度测量模块使用NTC热敏电阻（温度系数为负）检测温度的变化，因此其对温度变化非常灵敏。它可以将温度变化显示成电阻变化，利用这种特性，我们可以实现对当前环境温度值的获取。

实验原理：

温度传感器模块由 NTC 热敏电阻和定值电阻组成。热敏电阻的电阻值随环境温度变化而变化，使用 Steinhart-Hart 方程：

$$R = R_0 \exp \left[B \left(\frac{1}{T} - \frac{1}{T_0} \right) \right]$$

来得出热敏电阻的精确温度。在本实验中，我们使用模数转换器 PCF8591 将模拟信号转换为数字信号。模拟温度传感器的原理图如下图所示：



实验过程：

(1) 按照指导手册要求完成接线，其中模拟温度传感器的DO与树莓派的GPIO17引脚连接，AO与PCF8591模块的AIN0相连，芯片将从AIN0口读取数据，由于是输入模式（从AIN0），故将输入channel设置为0x00，具体原理见实验4的原理部分。

(2) 通过函数读取得到传感器模拟输出通过AD转化后的数字值: `analogVal = PCF8591.read(0)`

(3) 利用这个值计算热敏电阻的模拟电压值 `vr = 5 * float(analogVal) / 255`

(4) 由于电阻串联, 经过简单电路计算可得热敏电阻阻值 $R_t = 10000 * V_r / (5 - V_r)$

(5) 最后根据S-H方程, 可得到温度T:

$$R = R_0 \exp \left[B \left(\frac{1}{T} - \frac{1}{T_0} \right) \right]$$

T_0 : Standard Temperature 298.15 K (25 °C)

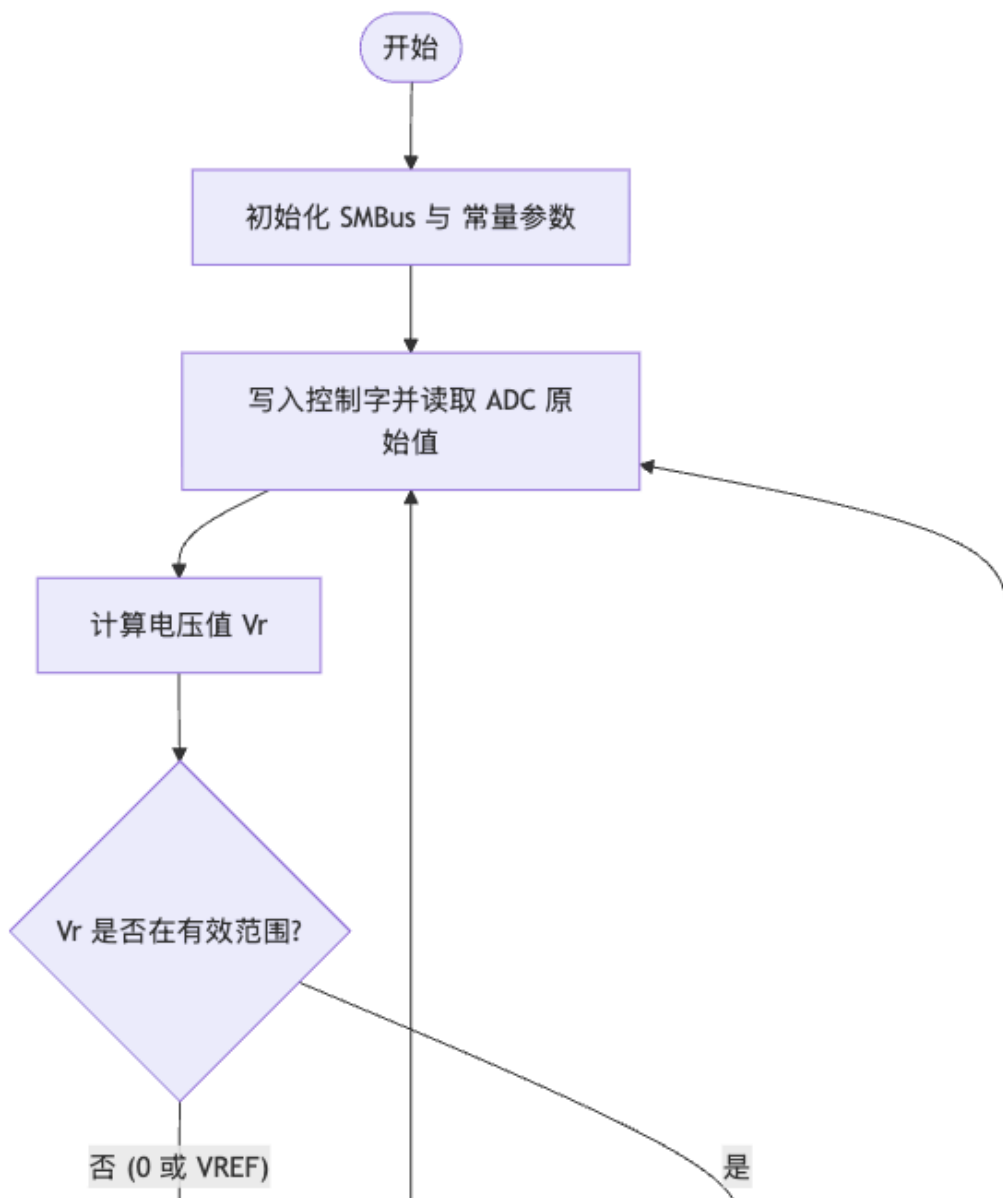
R_0 : Resistance at T_0 K (Ω)

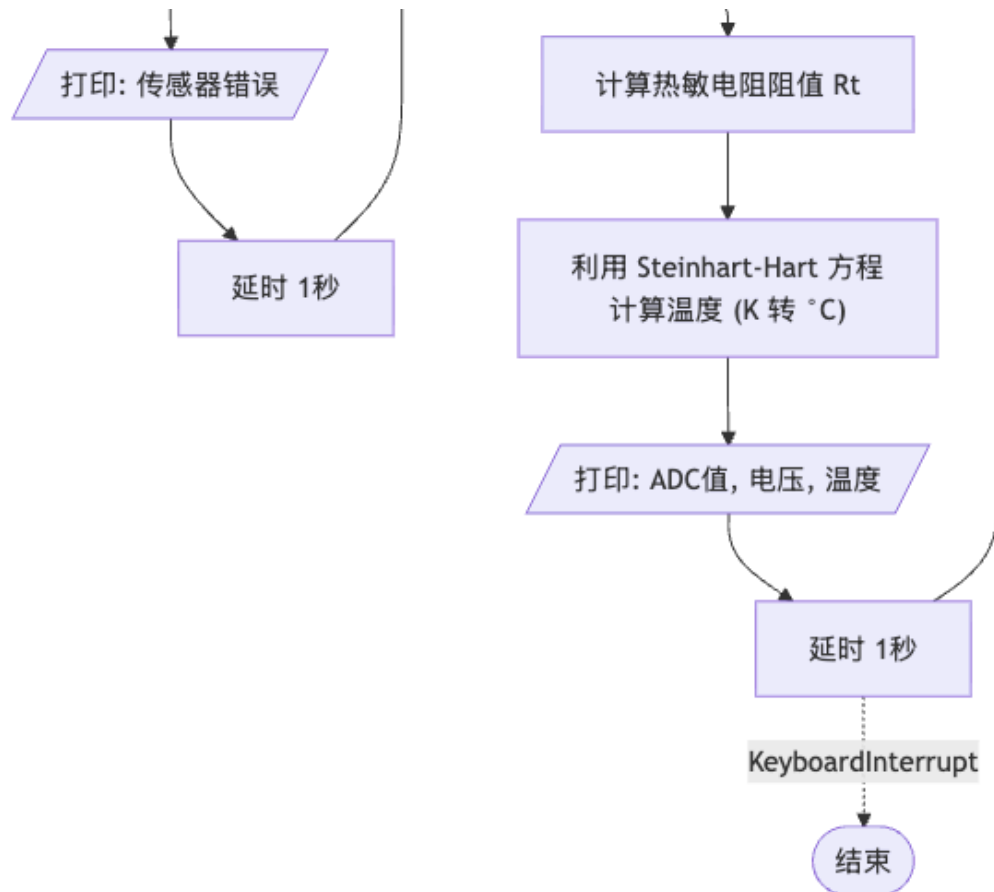
B: Thermistor Constant (K)

注意：由于温度传感器型号未知，本实验中， R_0 的值暂时选取为 10k，B 的值为 3950。

我们已经测出 R 的值，即为 R_t 。根据 Steinhart-Hart 方程即可推导出温度 T 的表达式。

程序框图：





程序代码：

```
import smbus
import time
import math

bus = smbus.SMBus(1)
DEVICE_ADDRESS = 0x48
CHANNEL = 0x00
VREF = 3.3

R0 = 10000
B = 3950
T0 = 298.15

def read_adc(channel):
    bus.write_byte(DEVICE_ADDRESS, channel)
    time.sleep(0.01)
    bus.read_byte(DEVICE_ADDRESS)
    return bus.read_byte(DEVICE_ADDRESS)

try:
    while True:
        analogVal = read_adc(CHANNEL)
        Vr = VREF * float(analogVal) / 255
        if Vr == 0 or Vr == VREF:
```

```

        print("Sensor error or out of range.")
        time.sleep(1)
        continue
    Rt = R0 * Vr / (VREF - Vr)
    tempK = 1 / (math.log(Rt / R0) / B + (1 / T0))
    tempC = tempK - 273.15
    print(f"ADC: {analogVal}, Voltage: {Vr:.2f} V, Temp: {tempC:.2f} °C")
    time.sleep(1)
except KeyboardInterrupt:
    print("End")

```

实验6:超声波测距实验

实验目的：

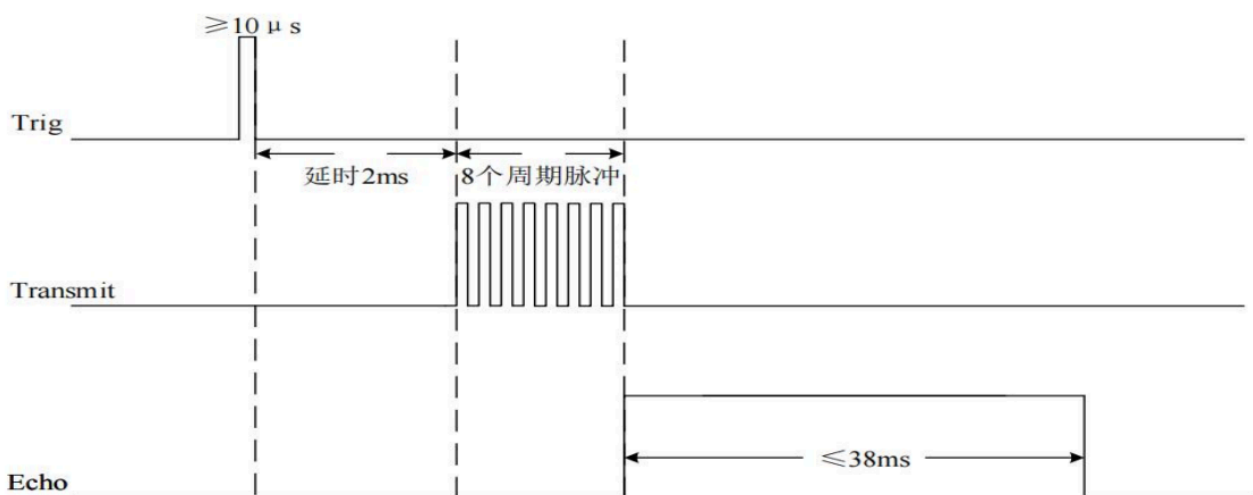
通过使用超声波测距模块，学习超声波传感器的触发和接受，并理解超声波测距的原理，最终获取障碍物与超声波传感器的距离。

实验原理：

超声波测距模块的工作原理如下：

1. 控制器给 Trig 引脚至少 $10\mu\text{s}$ （10 微秒）的高电平信号，触发传感器开始工作。
2. 超声波传感器经过一定的延时（2ms）后，发送 8 个 40KHz 的超声波脉冲信号。同时，Echo 引脚变为高电平。
3. 当检测到有回声信号返回或大于 38ms 时，Echo 引脚变为低电平。
4. 测试距离=（高电平时间*声速）/2

但当Echo引脚变为高电平时是5V，可是树莓派GPIO输入不可以超过3.3V，所以需要使用串联电阻进行分压。

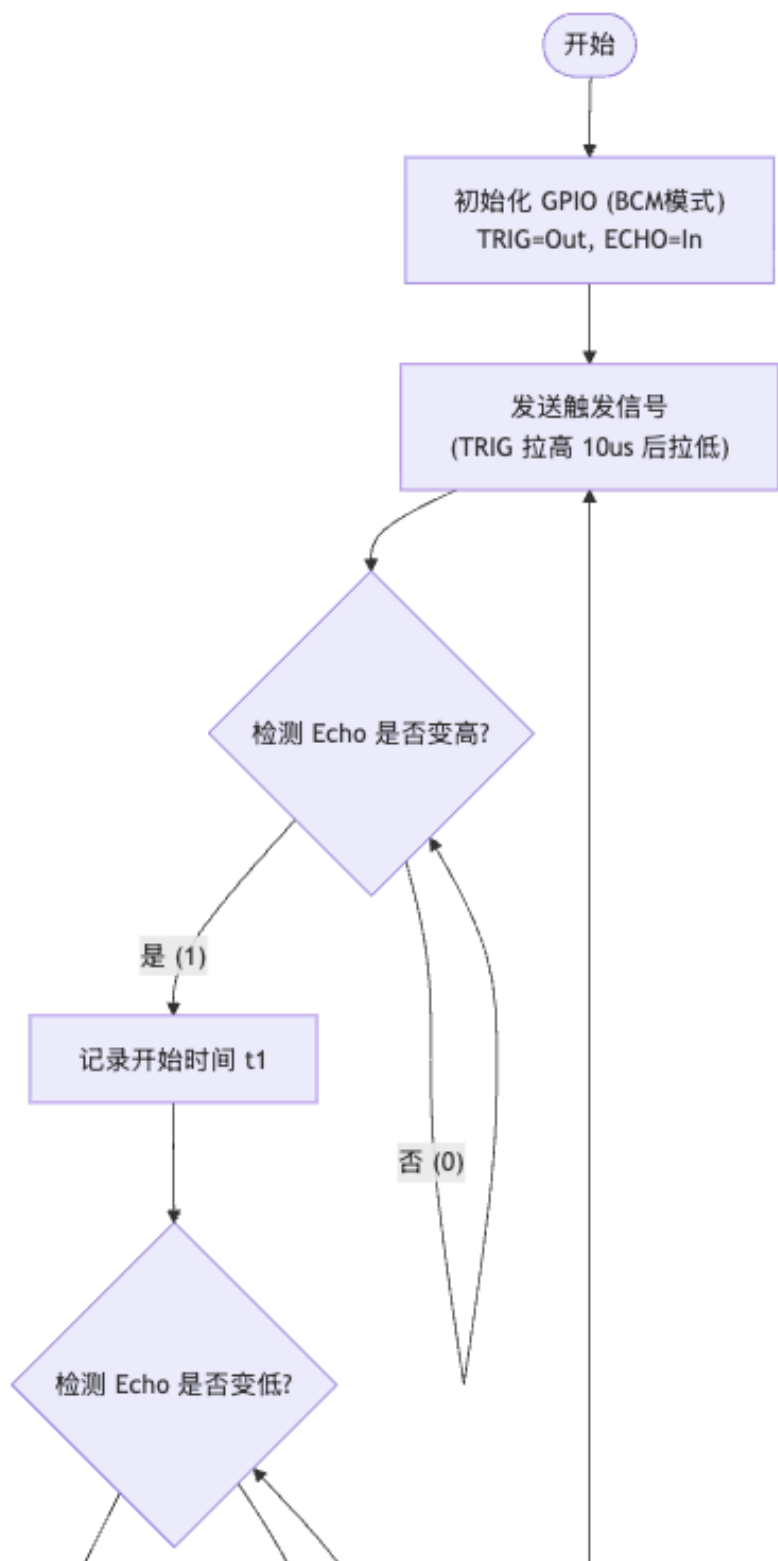


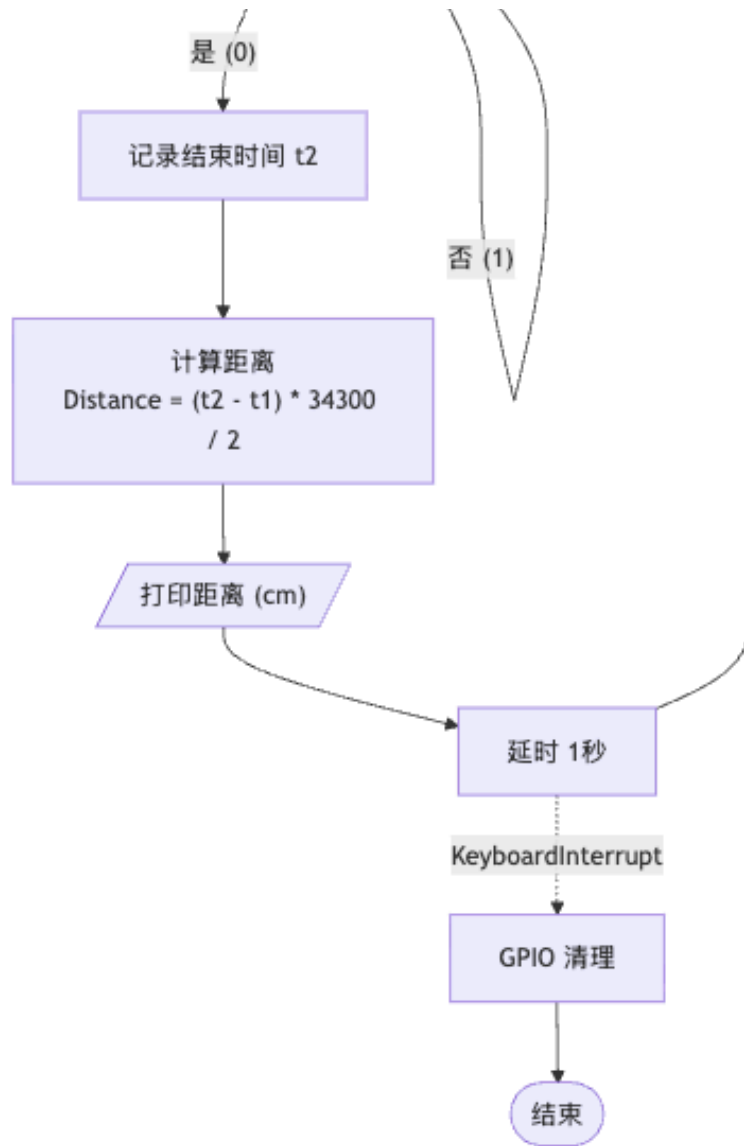
超声测距模块引脚波形图

实验过程：

- (1) 连接超声波测距模块：将 VCC 引脚连接到 5V 电源，Trig 引脚连接到GPIO17引脚，GND 引脚连接到树莓派的GND。
- (2) 对于Echo引脚，由于其需要输入给树莓派，所以需要进行串联分压，用330Ω和200Ω电阻进行串联分压，大致可以分得3.3V供给GPIO口。
- (3) 当选取测距物体时，需要注意物体的面积不能太小，不然很容易误识别，在本次试验中我选取手机作为测距物体。

程序框图：





程序代码：

```
import RPi.GPIO as GPIO
import time

TRIG = 17 # Trigger pin (output)
ECHO = 18 # Echo pin (input)

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

try:
    while True:
        GPIO.output(TRIG, GPIO.LOW)
        time.sleep(0.002)
        GPIO.output(TRIG, GPIO.HIGH)
        time.sleep(0.00001) # 10us
        GPIO.output(TRIG, GPIO.LOW)
```

```
while GPIO.input(ECHO) == 0:
    pass
t1 = time.time()

while GPIO.input(ECHO) == 1:
    pass
t2 = time.time()
distance = (t2 - t1) * 34300 / 2
print(f"Distance: {distance:.2f} cm")
time.sleep(1)
except KeyboardInterrupt:
    print("End")
finally:
    GPIO.cleanup()
```

实验7:蜂鸣器实验

实验目的:

通过编写代码，产生不同频率的脉冲信号，实现**两首音乐的播放**

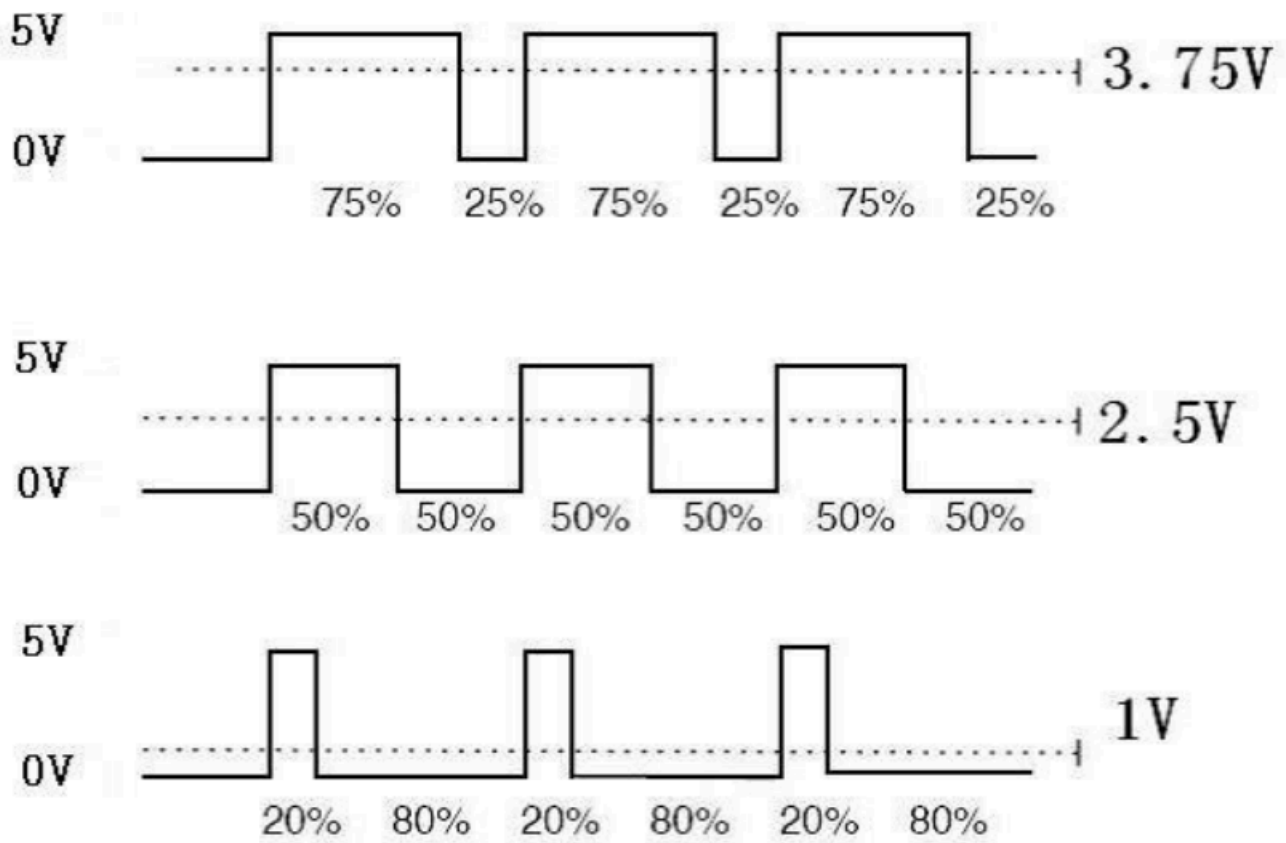
实验原理:

1. 蜂鸣器模块的工作原理

- a. 有源蜂鸣器：有源蜂鸣器内部包含一个振荡电路，可以将恒定的直流电源转化为一定频率的脉冲信号。这个振荡电路产生的脉冲信号的频率是固定的，一般为单一音调。有源蜂鸣器的工作原理比较简单，它通过输入一个低电平信号（通常为低电平触发），就可以发出特定频率的声音。在实验中，只需要控制蜂鸣器的开关状态即可发出声音，频率不可调。
- b. 无源蜂鸣器：无源蜂鸣器内部没有振荡电路，它需要接在特定的音频输出电路中才能发出声音。为了发出声音，需要通过脉冲宽度调制（PWM）或脉冲频率调制（PFM）的方式控制蜂鸣器的工作。在本实验中，我们使用PWM方式。

2. PFM方式和PWM方式的区别

PFM保证方波的脉冲宽度不变（高电平持续时间均为50%），而会改变脉冲的频率；PWM则正好相反，脉冲频率固定，改变占空比，实现输出电压的调制。

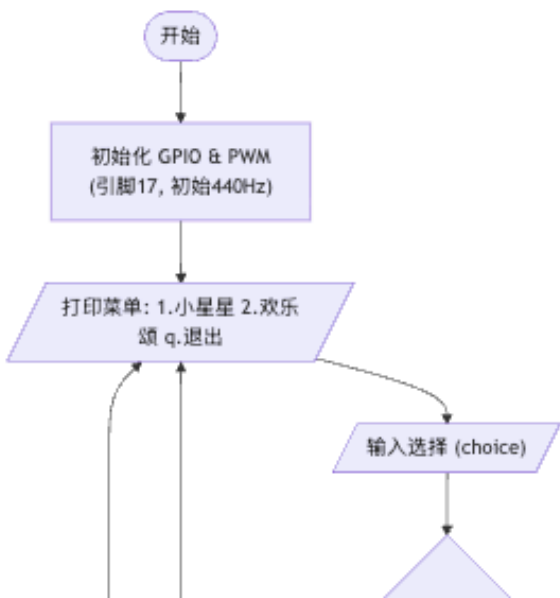


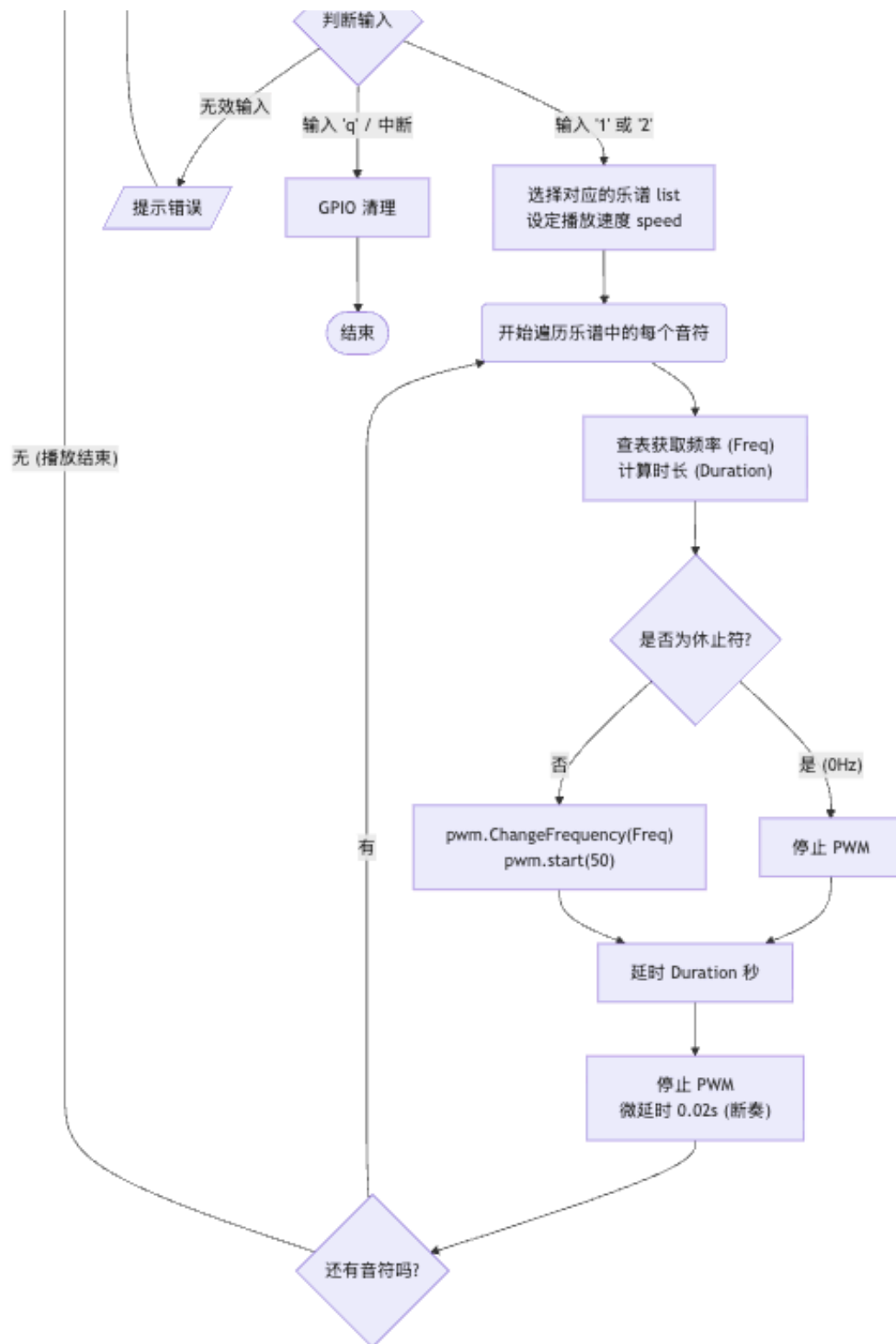
实验过程：

在本实验中，我们选取无源蜂鸣器以及PWM调制方式。

- (1) 将蜂鸣器的I/O口连接到GPIO17口，VCC连接到树莓派的3.3V引脚，GND与GND相连
- (2) 先设置440Hz的初始频率和50%的初始占空比，然后调节占空比，改变发声频率，构建一个映射字典，实现不同频率的PWM与音符之间的对应。
- (3) 根据所需要的歌曲进行音符播放。

程序框图：





程序代码：

```

import RPi.GPIO as GPIO
import time

buzzer_pin = 17

GPIO.setmode(GPIO.BCM)

```

```

GPIO.setup(buzzer_pin, GPIO.OUT)

pitch_to_freq = {
    'C4': 262, 'D4': 294, 'E4': 330, 'F4': 349, 'G4': 392, 'A4': 440, 'B4': 494,
    '0': 0
}

song_1_score = [
    ('C4', 1), ('C4', 1), ('G4', 1), ('G4', 1), ('A4', 1), ('A4', 1), ('G4', 2),
    ('F4', 1), ('F4', 1), ('E4', 1), ('E4', 1), ('D4', 1), ('D4', 1), ('C4', 2),
    ('G4', 1), ('G4', 1), ('F4', 1), ('F4', 1), ('E4', 1), ('E4', 1), ('D4', 2),
    ('G4', 1), ('G4', 1), ('F4', 1), ('F4', 1), ('E4', 1), ('E4', 1), ('D4', 2),
    ('C4', 1), ('C4', 1), ('G4', 1), ('G4', 1), ('A4', 1), ('A4', 1), ('G4', 2),
    ('F4', 1), ('F4', 1), ('E4', 1), ('E4', 1), ('D4', 1), ('D4', 1), ('C4', 2),
]

song_2_score = [
    ('E4', 1), ('E4', 1), ('F4', 1), ('G4', 1), ('G4', 1), ('F4', 1), ('E4', 1), ('D4',
1),
    ('C4', 1), ('C4', 1), ('D4', 1), ('E4', 1), ('E4', 1.5), ('D4', 0.5), ('D4', 2),
    ('E4', 1), ('E4', 1), ('F4', 1), ('G4', 1), ('G4', 1), ('F4', 1), ('E4', 1), ('D4',
1),
    ('C4', 1), ('C4', 1), ('D4', 1), ('E4', 1), ('D4', 1.5), ('C4', 0.5), ('C4', 2),
]

def play_song(pwm, score, speed=0.5):
    for note, beat in score:
        frequency = pitch_to_freq.get(note)
        duration = beat * speed
        if frequency == 0:
            pwm.stop()
        else:
            pwm.ChangeFrequency(frequency)
            pwm.start(50)
            time.sleep(duration)
            pwm.stop()
            time.sleep(0.02)

def main():
    pwm = GPIO.PWM(buzzer_pin, 440)
    while True:
        print("\n--- 蜂鸣器音乐播放器 ---")
        print("1. 播放《小星星》")
        print("2. 播放《欢乐颂》")
        print("q. 退出程序")
        choice = input("请输入你的选择 (1/2/q): ")
        if choice == '1':
            print("\n正在播放《小星星》...")
            play_song(pwm, song_1_score, speed=0.4)
            print("播放完毕。")
        elif choice == '2':
            print("\n正在播放《欢乐颂》...")

```

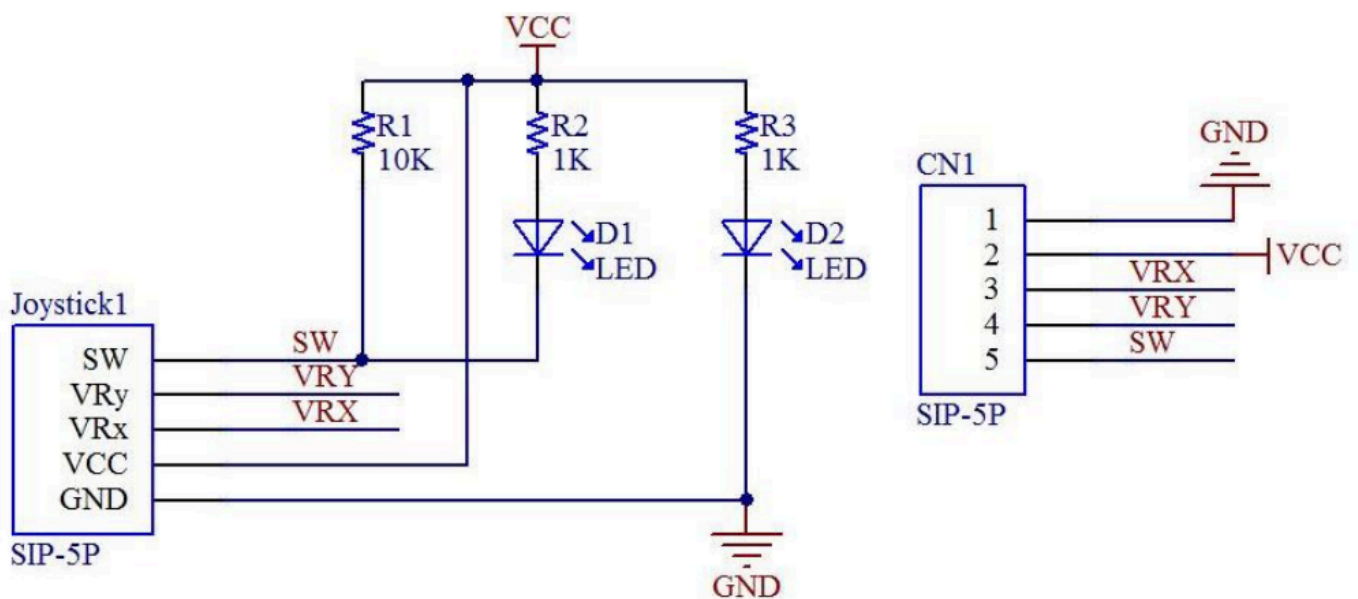
```

play_song(pwm, song_2_score, speed=0.35)
print("播放完毕。")
elif choice.lower() == 'q':
    print("正在退出程序...")
    break
else:
    print("无效的输入，请输入 1, 2 或 q。")
pwm.stop()

if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
        print("\n程序被用户中断")
    finally:
        GPIO.cleanup()
        print("GPIO 已清理，程序结束。")

```

实验目的:



实验原理：

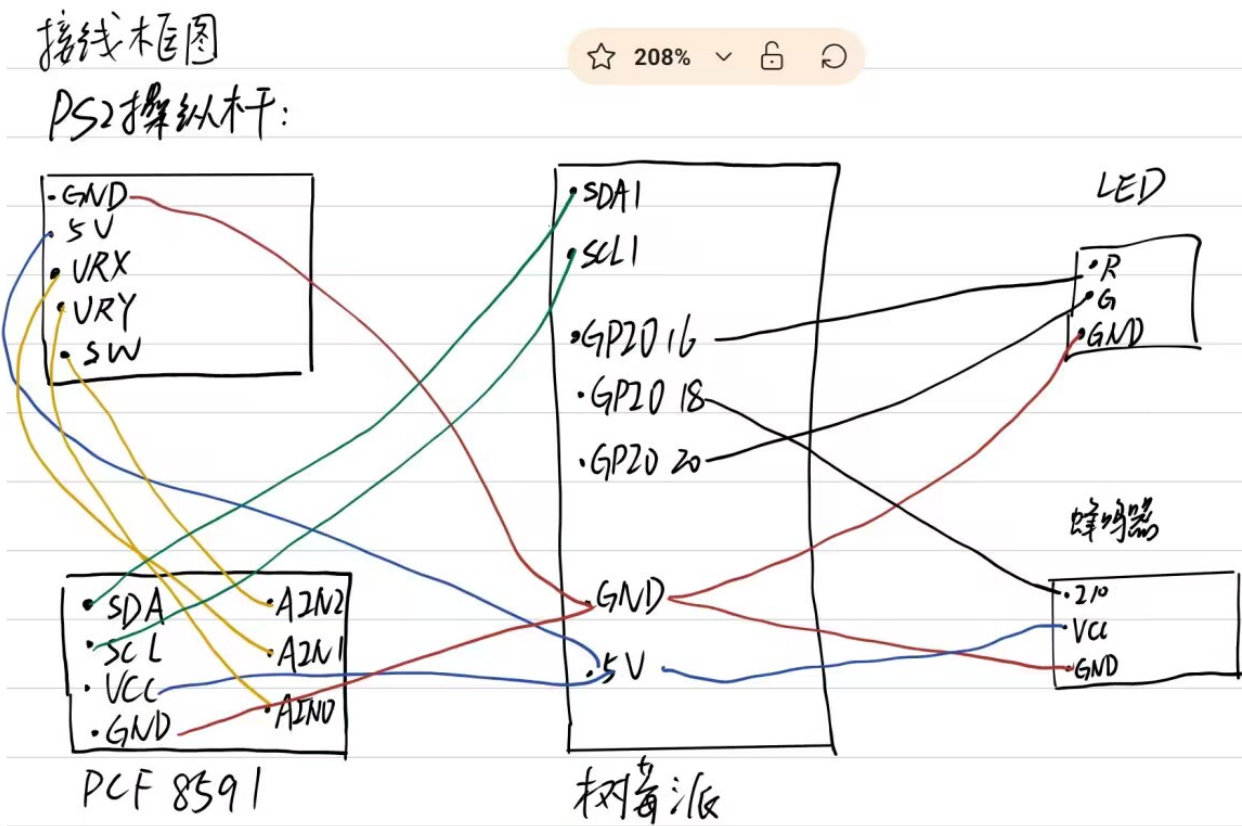
(2) 首先将PS2操纵杆连接到PCF8591芯片，再将芯片配置到树莓派的I2C总线实现通信，可以读取PCF8591对应的两个通道的数值，可以获取操纵杆在X和Y方向上的位置信息，将这个位置和亮度相对应。

(3) PS2操纵杆z方向还可以产生数字量，但是在本实验中没有焊接电阻，无法产生稳定的数字值，不可稳定测量，故不进行蜂鸣实验。

实验过程：

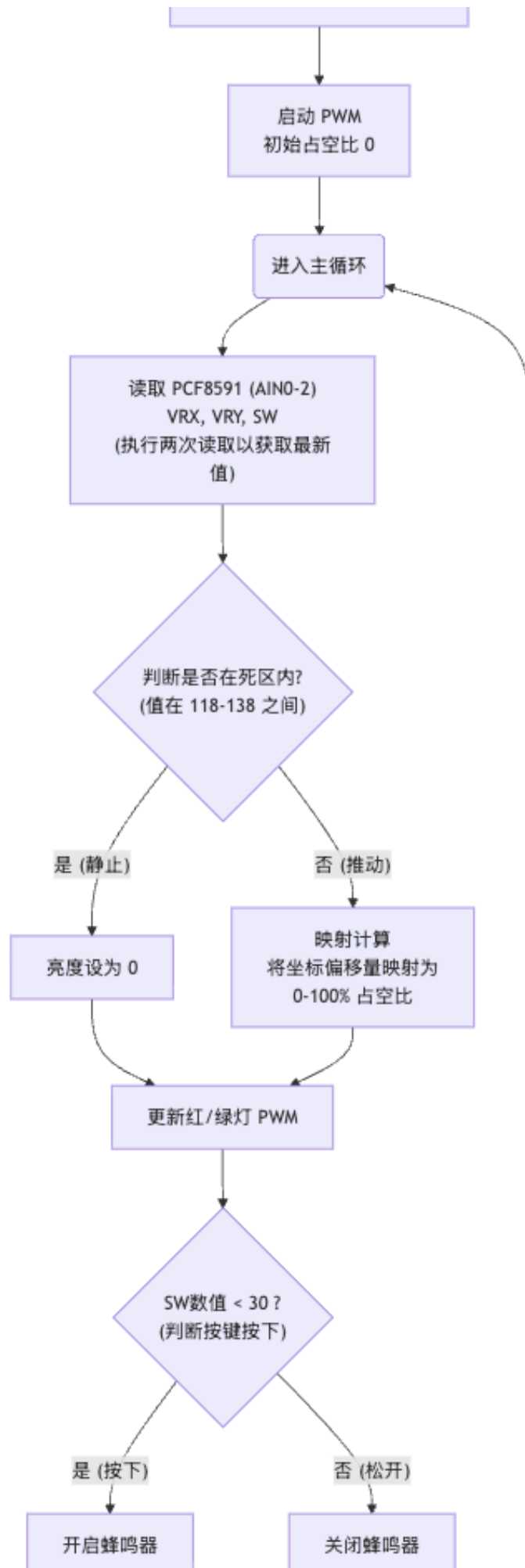
- (1) 设置PCF8591的地址为 0x48，然后把红灯灯管接到GPIO16口，绿灯灯管灯管接到GPIO20口，蜂鸣器接到GPIO18口。PCF8591的AIN0-2口分别对应PS2操纵杆的VRX、VRY和SW，将模拟电压转换为0-255的数字信号。
- (2) 由于PCF8591的数据残留特性，需要连续执行两次读取，取第二次的值。
- (3) 亮度调节：由于物理摇杆并不精确，设置一个10的死区防止抖动。随后在PS2操纵杆的X、Y方向进行映射，数值从128-255时，亮度从0变到100（这里我并没有把摇杆停留在中立位时作为50%亮度而是0%）
- (4) 蜂鸣器：Z方向是一个数字信号，但是还是将其连接到AIN2，并将30作为阈值（实际上SW按下时应该是0），如果小于30就会启动蜂鸣器。

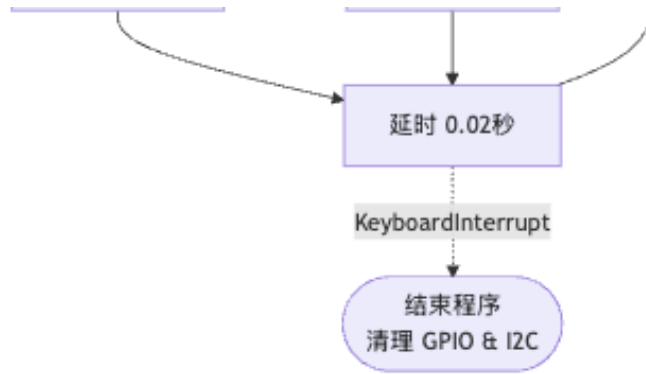
硬件接线图：



程序框图：







程序代码：

```
import time
import smbus2
import RPi.GPIO as GPIO

I2C_BUS = 1
PCF8591_ADDR = 0x48

GPIO_RED = 16
GPIO_GREEN = 20
GPIO_BUZZER = 18

CHANNEL_VRX = 0
CHANNEL_VRY = 1
CHANNEL_SW = 2

CENTER_VALUE = 128
DEADZONE = 10
SW_PRESSED_THRESHOLD = 30

LED_PWM_FREQ = 1000
BUZZER_FREQ = 2000

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)

GPIO.setup(GPIO_RED, GPIO.OUT)
GPIO.setup(GPIO_GREEN, GPIO.OUT)
GPIO.setup(GPIO_BUZZER, GPIO.OUT)

pwm_red = GPIO.PWM(GPIO_RED, LED_PWM_FREQ)
pwm_green = GPIO.PWM(GPIO_GREEN, LED_PWM_FREQ)
pwm_buzzer = GPIO.PWM(GPIO_BUZZER, BUZZER_FREQ)

pwm_red.start(0)
pwm_green.start(0)

buzzer_is_on = False
```

```

bus = smbus2.SMBus(I2C_BUS)

def read_adc(channel):
    if channel < 0 or channel > 3:
        return 0
    control_byte = 0x40 | channel
    bus.write_byte(PCF8591_ADDR, control_byte)
    bus.read_byte(PCF8591_ADDR)
    value = bus.read_byte(PCF8591_ADDR)
    return value

def map_range(value, in_min, in_max, out_min, out_max):
    value = max(in_min, min(in_max, value))
    return (value - in_min) * (out_max - out_min) / (in_max - in_min) + out_min

def calculate_brightness(axis_value, reverse=False):
    if abs(axis_value - CENTER_VALUE) < DEADZONE:
        return 0
    if reverse:
        if axis_value < CENTER_VALUE:
            duty = map_range(axis_value, 0, CENTER_VALUE - DEADZONE, 100, 0)
        else:
            duty = 0
    else:
        if axis_value > CENTER_VALUE:
            duty = map_range(axis_value, CENTER_VALUE + DEADZONE, 255, 0, 100)
        else:
            duty = 0
    return max(0, min(100, duty))

try:
    while True:
        vrx = read_adc(CHANNEL_VRX)
        vry = read_adc(CHANNEL_VRY)
        sw = read_adc(CHANNEL_SW)
        red_duty = calculate_brightness(vry, reverse=False)
        pwm_red.ChangeDutyCycle(red_duty)
        green_duty = calculate_brightness(vrx, reverse=True)
        pwm_green.ChangeDutyCycle(green_duty)
        if sw <= SW_PRESSED_THRESHOLD:
            if not buzzer_is_on:
                pwm_buzzer.start(50)
                buzzer_is_on = True
            else:
                if buzzer_is_on:
                    pwm_buzzer.stop()
                    buzzer_is_on = False
        time.sleep(0.02)

except KeyboardInterrupt:
    pass

```

```
finally:
    pwm_red.stop()
    pwm_green.stop()
    if buzzer_is_on:
        pwm_buzzer.stop()
    GPIO.cleanup()
    bus.close()
```

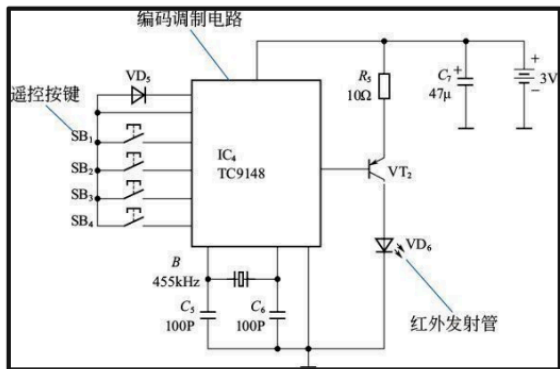
实验9:红外遥控实验

实验目的：

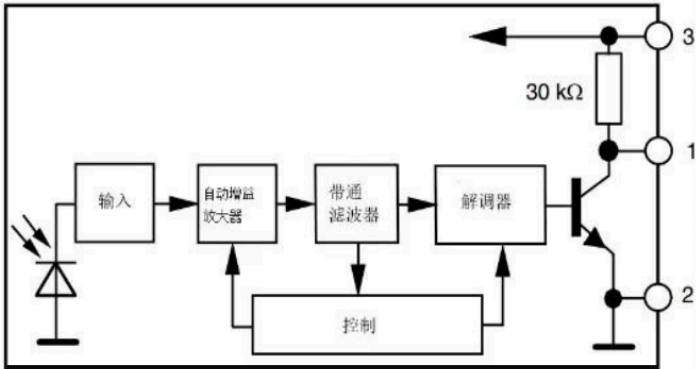
本实验首先进行采集数据，随后用红外遥控器发射信号。通过lirc库读取遥控器按钮返回的红外信号，将它们转换为按钮值。

实验原理：

(1) 红外通信是利用红外技术实现两点间的近距离保密通信和信息转发。一般由红外发射和接收系统两部分组成。发射系统对一个红外辐射源进行调制后发射红外信号，而接收系统用光学装置和红外探测器进行接收，就构成红外通信系统。



红外遥控发射器



红外接收器

红外系统工作流程电路图

(2) `lirc` (Linux Infrared Remote Control) 库是一个用于 Linux 系统的红外遥控器接口库。它允许我们在 Linux 系统上轻松地捕获和解码红外信号，以便识别遥控器按键并执行相应的操作。

(3) 而在树莓派正式接受前，需要通过安装库、配置、录制按键来完成红外接收器和发射器的配对。

实验过程：

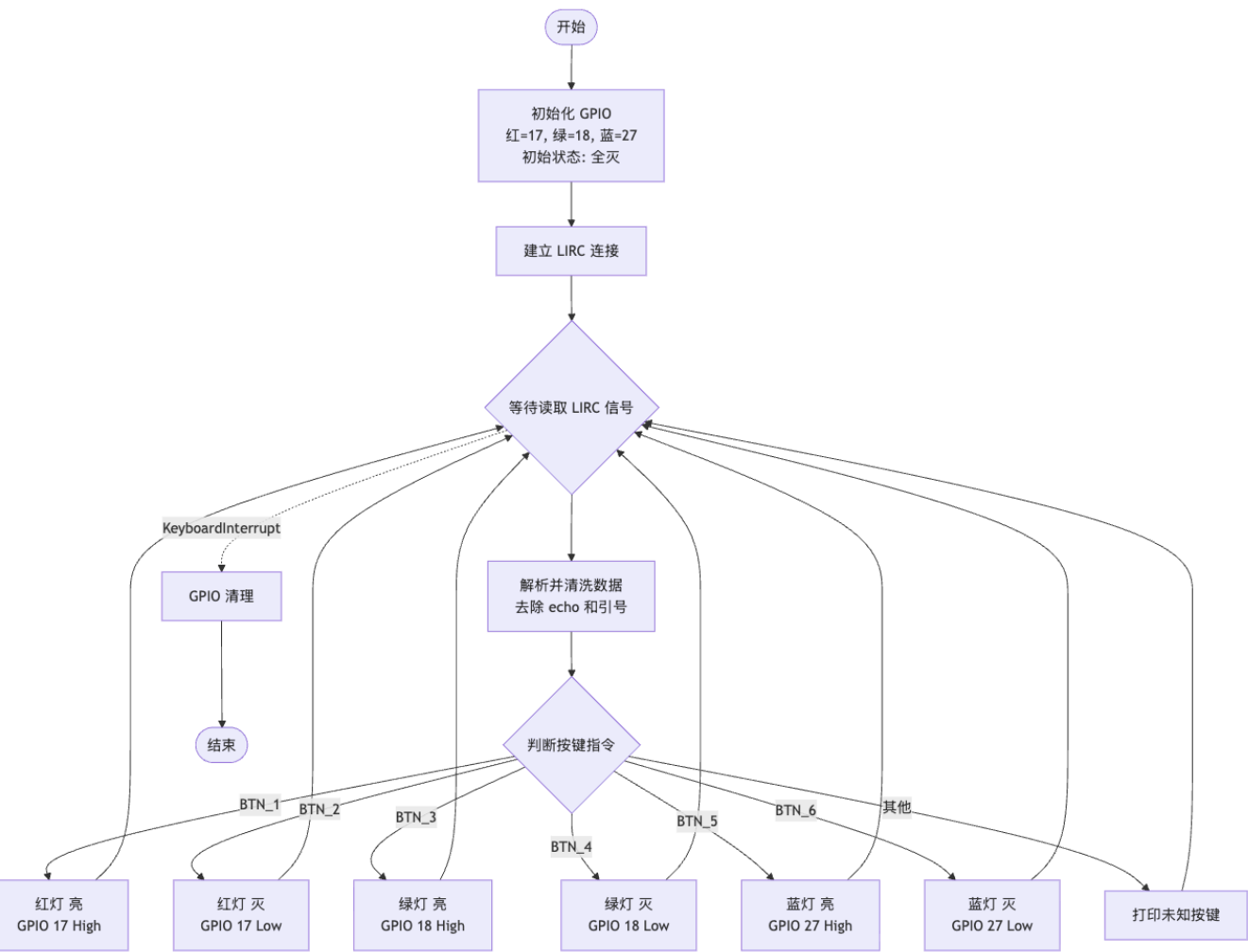
- 按照实验手册的指示，完成树莓派上的lirc库的安装和配置。
- 按照实验手册的指示，完成红外遥控器按键的录制。本实验中我录入了BTN_1-BTN_6六个按键（后续发现如果要弹奏歌曲需要至少录入7个）。

(3) 完成任务1:展示出LED灯的三种状态

首先将红、绿、蓝三色灯管分别接到GPIO17、18、27引脚。随后通过lirc库捕捉红外信号，并将按键与灯光状态相对应：

BTN_1	红灯亮
BTN_2	红灯灭
BTN_3	绿灯亮
BTN_4	绿灯灭
BTN_5	蓝灯亮
BTN_6	蓝灯灭

程序框图：



程序代码：

```
import lirc
import RPi.GPIO as GPIO
import time

RED_PIN = 17
GREEN_PIN = 18
```

```

BLUE_PIN = 27

GPIO.setmode(GPIO.BCM)
GPIO.setup(RED_PIN, GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(GREEN_PIN, GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(BLUE_PIN, GPIO.OUT, initial=GPIO.LOW)

def pasreset(data):
    key = data.strip().replace('echo ', '').replace("'", '')
    if key == 'BTN_1':
        print("红灯亮")
        GPIO.output(RED_PIN, GPIO.HIGH)
    elif key == 'BTN_2':
        print("红灯灭")
        GPIO.output(RED_PIN, GPIO.LOW)
    elif key == 'BTN_3':
        print("绿灯亮")
        GPIO.output(GREEN_PIN, GPIO.HIGH)
    elif key == 'BTN_4':
        print("绿灯灭")
        GPIO.output(GREEN_PIN, GPIO.LOW)
    elif key == 'BTN_5':
        print("蓝灯亮")
        GPIO.output(BLUE_PIN, GPIO.HIGH)
    elif key == 'BTN_6':
        print("蓝灯灭")
        GPIO.output(BLUE_PIN, GPIO.LOW)
    else:
        print(f"收到未知按键: {key}")

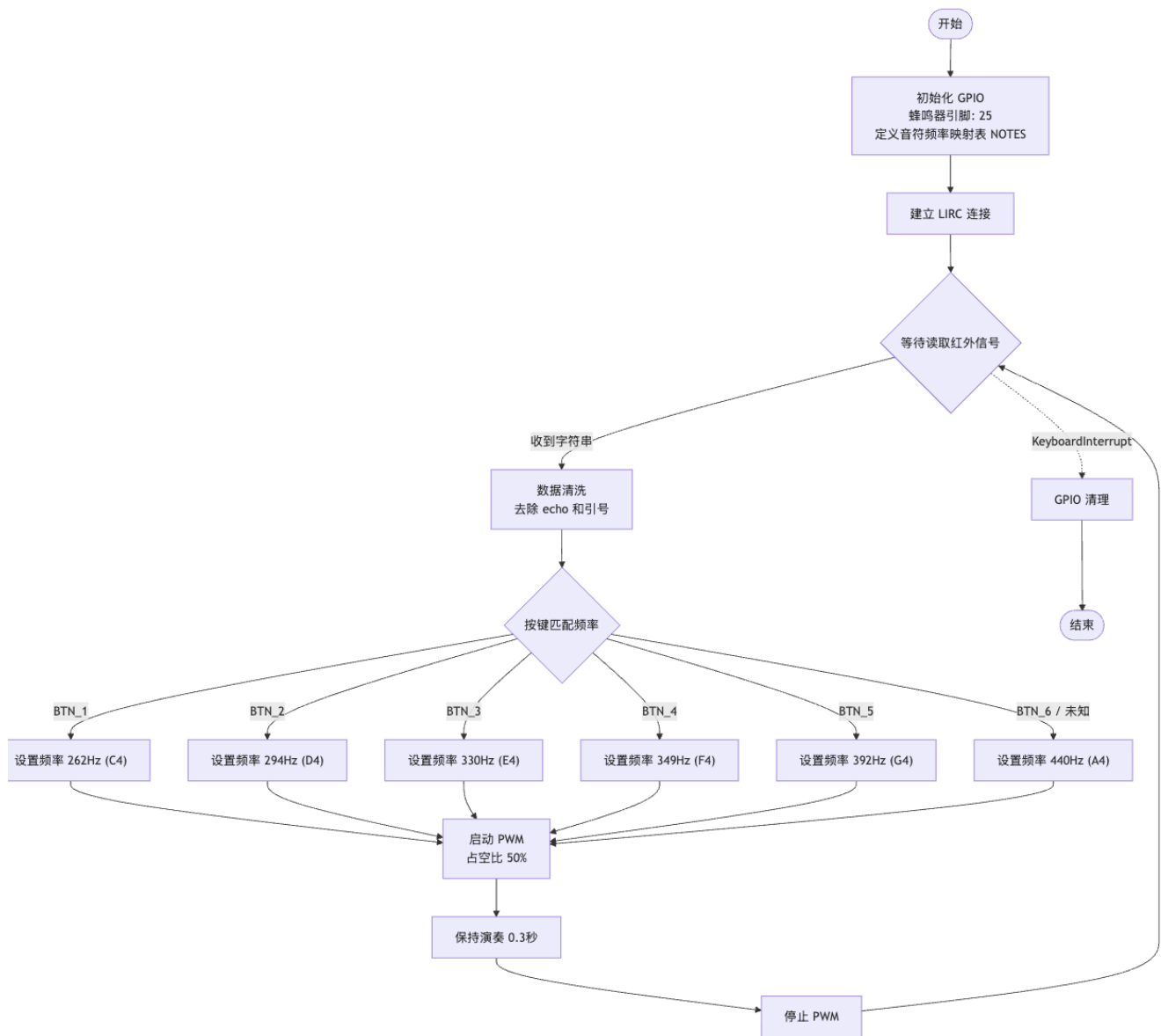
try:
    with lirc.LircdConnection("test.py",) as conn:
        while True:
            string = conn.readline().strip()
            pasreset(string)
except KeyboardInterrupt:
    print("程序终止")
finally:
    GPIO.cleanup()
    print("GPIO已清理")

```

(4) 完成任务2:用遥控器演奏《小星星》

这个任务中，我把蜂鸣器接入GPIO25引脚。由于我少录入了一个按键，就将1-6按钮映射为了C-G音符，每个音符调节其PWM占空比完成频率区分，后续按钮按下时会产生不同频率的音符，也就实现了歌曲演奏。

程序框图：



程序代码：

```

import lirc
import RPi.GPIO as GPIO
import time

BUZZER_PIN = 25
GPIO.setmode(GPIO.BCM)
GPIO.setup(BUZZER_PIN, GPIO.OUT, initial=GPIO.LOW)

# 音符频率 (C D E F G A)
NOTES = {
    'BTN_1': 262,    # C4
    'BTN_2': 294,    # D4
    'BTN_3': 330,    # E4
    'BTN_4': 349,    # F4
    'BTN_5': 392,    # G4

```

```

    'BTN_6': 440    # A4
}

def play_note(btn, duration=0.3):
    freq = NOTES.get(btn, NOTES['BTN_6'])
    buzzer = GPIO.PWM(BUZZER_PIN, freq)
    buzzer.start(50)
    time.sleep(duration)
    buzzer.stop()

def pasreset(data):
    key = data.strip().replace('echo ', '').replace("'", '')
    play_note(key)

try:
    with lirc.LircdConnection("test.py",) as conn:
        while True:
            string = conn.readline().strip()
            pasreset(string)
except KeyboardInterrupt:
    print("程序终止")
finally:
    GPIO.cleanup()
    print("GPIO已清理")

```

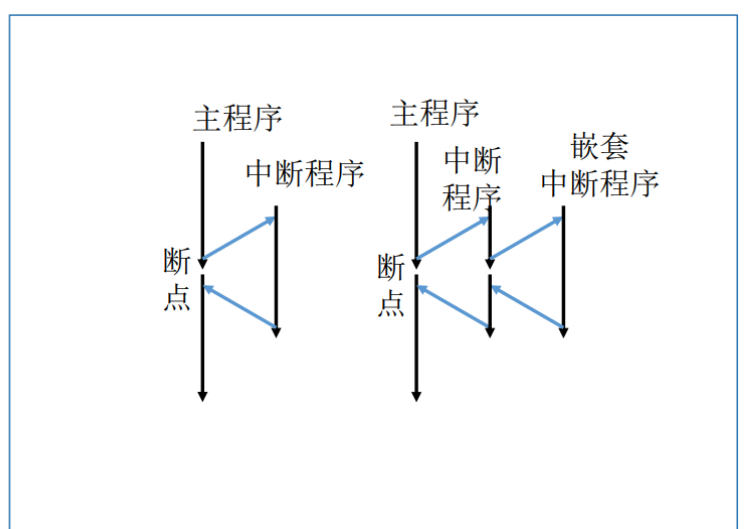
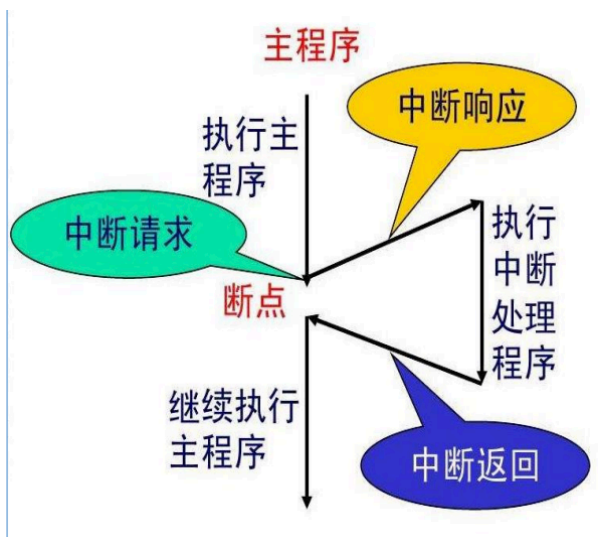
实验10:中断实验

实验目的：

外部中断是指在程序执行过程中，由外部输入触发的一种机制，用于暂停当前程序的执行，转而执行其他指定任务。外部中断的存在主要是为了处理紧急事件，例如硬件设备的输入、定时器的到期等。树莓派也可实现类似的功能。本次实验任务为通过中断机制在树莓派上实现对外接设备的及时响应。

实验原理：

在之前STM32的课程中，已经学习了“中断”的作用，具体如下图：



STM32中断的流程

中断的处理是一套标准化流程，确保CPU能“有序暂停、处理、恢复”

但是在树莓派中，中断控制程序是被封装在 `callback`接口 中的：

树莓派中断函数

➤ `add_event_detect(channel, edge, callback=None, bouncetime=None)`

Channel:需要检测状态变化的GPIO引脚

Edge: 指定要监测的边沿类型，可以是GPIO.RISING，GPIO.FALLING，或者GPIO.BOTH

Callback:可选参数，指定状态变化时需要执行的回调函数

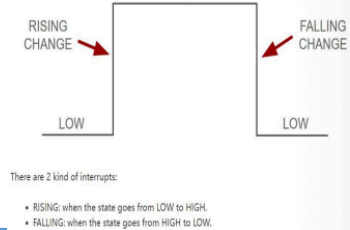
Bouncetime:可选参数，用于消除抖动效应，指定两次状态变化之间的最小间隔(单位为ms)

-->异步非阻塞（程序继续跑，中断触发后回调），CPU占用极低

➤ `GPIO.wait_for_edge(channel, GPIO.RISING)`

主要用于在检测到边缘前阻止程序运行(占用CPU资源少)

-->同步阻塞（程序暂停等中断），CPU占用极低（%0 级别）

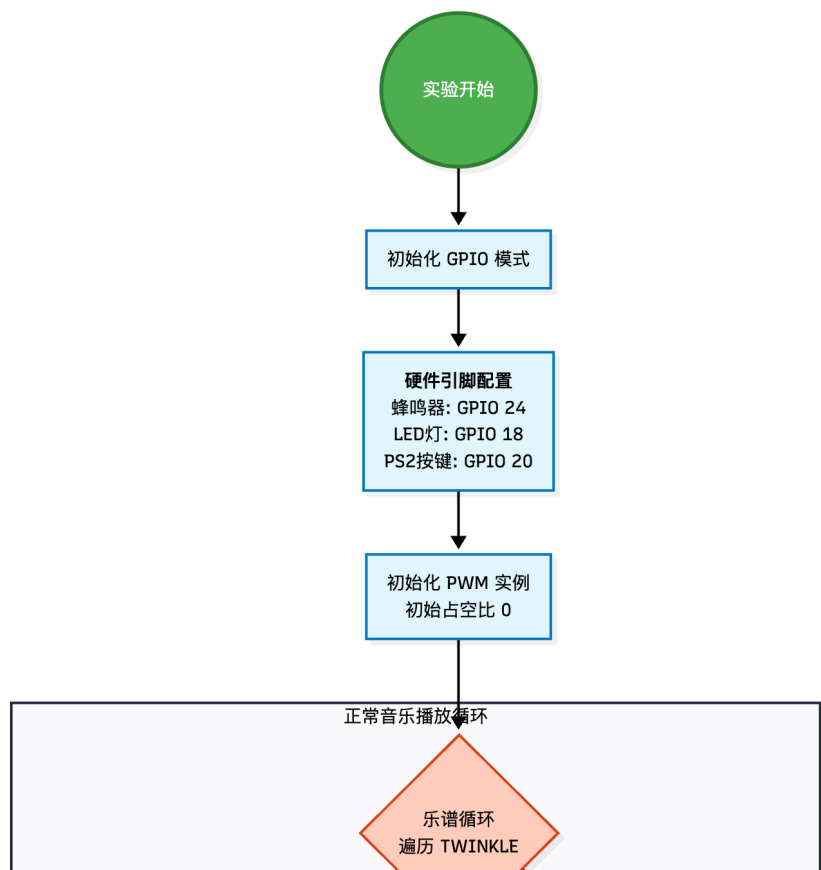


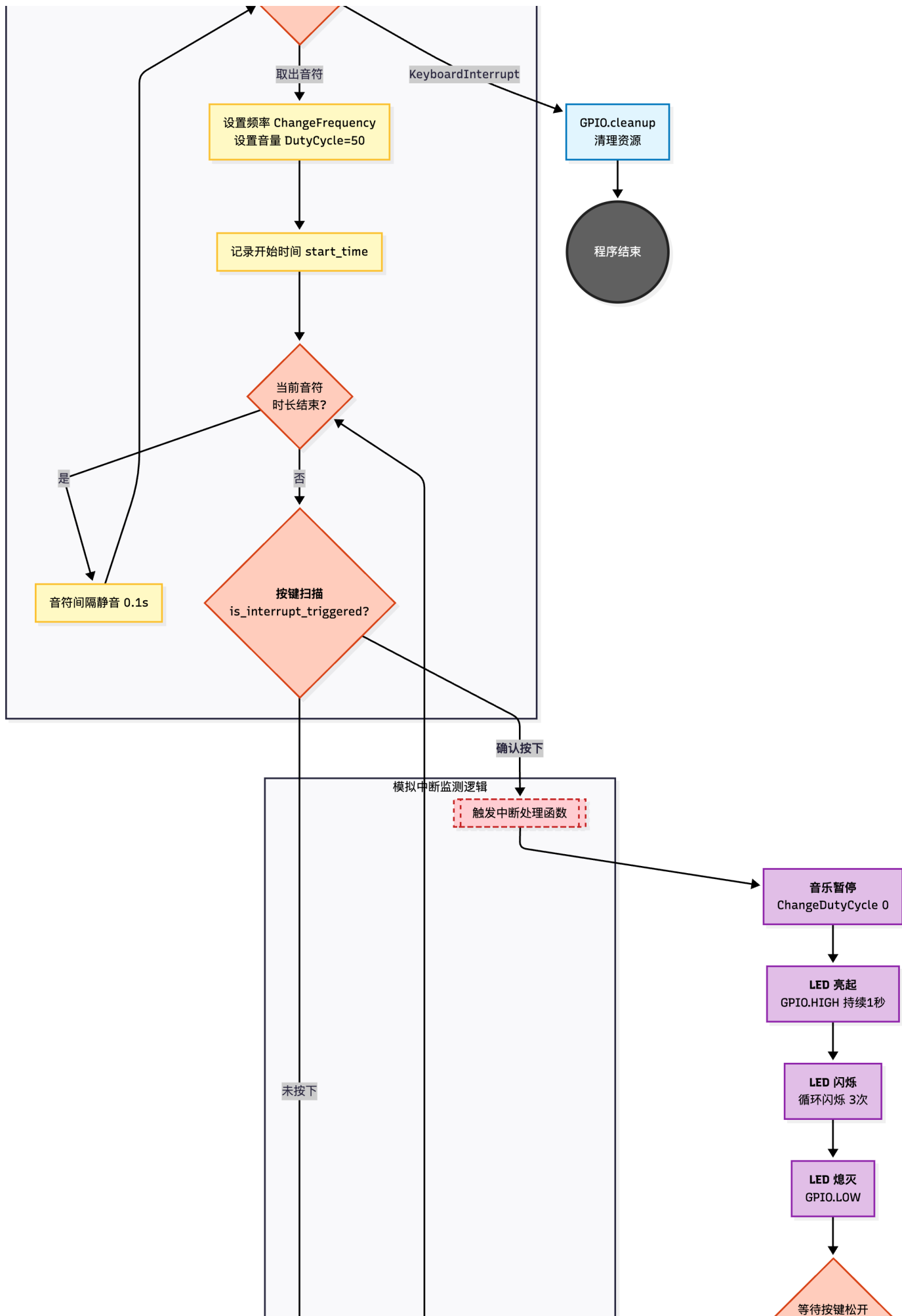
实验过程：

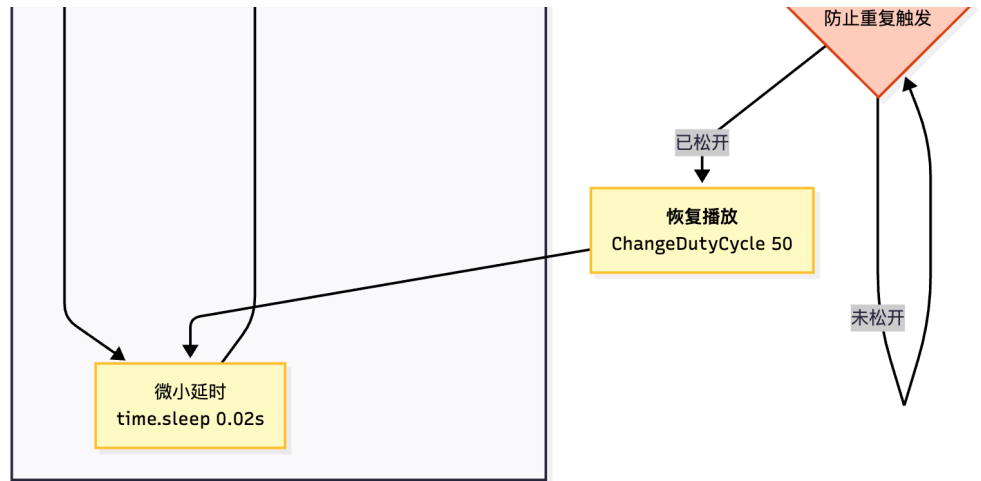
本实验要求完成音乐播放中断功能：在音乐正常播放时按下按键，音乐播放中断，LED灯亮起，待 LED 灯闪烁几秒后自动继续播放之前的音乐。

- (1) 之前学习了红外遥控和PS2操纵杆两种按键，这里我选择了使用PS2操纵杆，将其按键接到GPIO20.
- (2) 并且要求蜂鸣器作为播放音乐的工具（接GPIO24）以及LED灯作为中断指示的标志（接GPIO18）

程序框图：







程序代码：

```

import RPi.GPIO as GPIO
import time
BUZZER_PIN = 25
LED_PIN = 18
SWITCH_PIN = 20
MELODY = [262, 294, 330, 349, 392, 440, 494, 523]
TWINKLE = [
    (262, 0.4), (262, 0.4), (392, 0.4), (392, 0.4), (440, 0.4), (440, 0.4), (392, 0.8),
    (349, 0.4), (349, 0.4), (330, 0.4), (330, 0.4), (294, 0.4), (294, 0.4), (262, 0.8),
    (392, 0.4), (392, 0.4), (349, 0.4), (349, 0.4), (330, 0.4), (330, 0.4), (294, 0.8),
    (392, 0.4), (392, 0.4), (349, 0.4), (349, 0.4), (330, 0.4), (330, 0.4), (294, 0.8),
    (262, 0.4), (262, 0.4), (392, 0.4), (392, 0.4), (440, 0.4), (440, 0.4), (392, 0.8),
    (349, 0.4), (349, 0.4), (330, 0.4), (330, 0.4), (294, 0.4), (294, 0.4), (262, 0.8)
]

TRIGGER_CONFIRM = 3
SAMPLE_DELAY = 0.01

def setup():
    GPIO.setmode(GPIO.BCM)
    GPIO.setwarnings(False)
    GPIO.setup(LED_PIN, GPIO.OUT)
    GPIO.output(LED_PIN, GPIO.LOW)
    GPIO.setup(BUZZER_PIN, GPIO.OUT)
    GPIO.setup(SWITCH_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)

def is_button_pressed():
    return GPIO.input(SWITCH_PIN) == GPIO.LOW

def wait_for_button_release():
    while is_button_pressed():
        time.sleep(SAMPLE_DELAY)

def is_interrupt_triggered():
    consecutive = 0
    max_checks = TRIGGER_CONFIRM * 2
    for _ in range(max_checks):

```

```

        if is_button_pressed():
            consecutive += 1
            if consecutive >= TRIGGER_CONFIRM:
                return True
        else:
            consecutive = 0
        time.sleep(SAMPLE_DELAY)
    return False

def interrupt_handler(buzzer_pwm):
    print("\n>>> [中断触发] 按键被按下! 音乐暂停...")
    buzzer_pwm.ChangeDutyCycle(0)
    GPIO.output(LED_PIN, GPIO.HIGH)
    time.sleep(1)
    print(">>> LED 闪烁中...")
    for _ in range(3):
        GPIO.output(LED_PIN, GPIO.LOW)
        time.sleep(0.1)
        GPIO.output(LED_PIN, GPIO.HIGH)
        time.sleep(0.1)
    GPIO.output(LED_PIN, GPIO.LOW)
    print(">>> 请松开按键以继续...")
    wait_for_button_release()
    print(">>> [中断结束] 恢复音乐播放\n")
    buzzer_pwm.ChangeDutyCycle(50)

def play_tone(buzzer_pwm, freq, duration):
    buzzer_pwm.ChangeFrequency(freq)
    buzzer_pwm.ChangeDutyCycle(50)
    start_time = time.time()
    while (time.time() - start_time) < duration:
        if is_interrupt_triggered():
            interrupt_handler(buzzer_pwm)
            buzzer_pwm.ChangeFrequency(freq)
            time.sleep(0.02)

def main():
    setup()
    buzzer_pwm = GPIO.PWM(BUZZER_PIN, 440)
    buzzer_pwm.start(0)
    print("程序已启动。")
    print("正常播放小星星, 按下按键可触发中断效果。")
    try:
        while True:
            for freq, dur in TWINKLE:
                play_tone(buzzer_pwm, freq, dur)
                buzzer_pwm.ChangeDutyCycle(0)
                start_gap = time.time()
                while (time.time() - start_gap) < 0.1:
                    if is_interrupt_triggered():
                        interrupt_handler(buzzer_pwm)
                        time.sleep(0.02)

```



```
except KeyboardInterrupt:
    print("\n退出程序")
finally:
    buzzer_pwm.stop()
    GPIO.cleanup()

if __name__ == "__main__":
    main()
```